

# CONDYSOS

Sistema de Administracion Residencial

## MANUAL TECNICO

|                 |                                                                                       |
|-----------------|---------------------------------------------------------------------------------------|
| Documento       | Manual Tecnico                                                                        |
| Sistema         | CONDYSOS - administracion residencial                                                 |
| Stack           | Django 6.1 + Django REST Framework + Channels                                         |
| Version         | 1.0                                                                                   |
| Fecha           | Octubre 2026                                                                          |
| Repositorio     | <a href="https://github.com/JMJ0331/condosys">https://github.com/JMJ0331/condosys</a> |
| Rama de trabajo | desarrollo                                                                            |
| Licencia        | Proyecto academico / uso interno                                                      |

Documento generado a partir del codigo fuente del proyecto.  
Resumen de arquitectura, modelos de datos, API REST, permisos, frontend y procedimientos de operacion.

# Indice

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1. Introduccion</b>                                    | <b>7</b>  |
| 1.1 Proposito del documento                               | 7         |
| 1.2 Que es CONDYSOS                                       | 7         |
| 1.3 Alcance funcional                                     | 7         |
| 1.4 Perfiles de usuario                                   | 8         |
| 1.5 Convenciones del documento                            | 8         |
| <b>2. Stack tecnologico y dependencias</b>                | <b>9</b>  |
| 2.1 Lenguaje y runtime                                    | 9         |
| 2.2 Dependencias del proyecto                             | 9         |
| 2.3 Componentes principales                               | 10        |
| 2.4 Dependencias declaradas pero no activas               | 10        |
| <b>3. Arquitectura del proyecto</b>                       | <b>12</b> |
| 3.1 Vision general                                        | 12        |
| 3.2 Estructura de directorios                             | 12        |
| 3.3 El patron por aplicacion                              | 13        |
| 3.4 Capa de presentacion: dos superficies en la misma app | 14        |
| 3.5 Patrones de implementacion recurrentes                | 14        |
| Visibilidad por usuario en los listados                   | 14        |
| Serializers diferenciados para listado y detalle          | 15        |
| Cambios de estado auditados                               | 15        |
| Comprobante en PDF                                        | 15        |
| <b>4. Instalacion, ejecucion y operacion</b>              | <b>17</b> |
| 4.1 Requisitos previos                                    | 17        |
| 4.2 Instalacion desde cero                                | 17        |
| Paso 1 - Clonar el repositorio                            | 17        |
| Paso 2 - Crear el entorno virtual                         | 17        |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| Paso 3 - Instalar dependencias . . . . .                            | 17        |
| Paso 4 - Configurar el archivo .env . . . . .                       | 17        |
| Paso 5 - Aplicar migraciones . . . . .                              | 18        |
| Paso 6 - Cargar datos de ejemplo (opcional) . . . . .               | 18        |
| Paso 7 - Arrancar el servidor de desarrollo . . . . .               | 19        |
| 4.3 Comandos de uso frecuente . . . . .                             | 19        |
| 4.4 Flujo de trabajo con git . . . . .                              | 19        |
| 4.5 Rutas principales del sistema . . . . .                         | 20        |
| <b>5. Usuarios, roles y permisos . . . . .</b>                      | <b>21</b> |
| 5.1 Modelo de usuario . . . . .                                     | 21        |
| 5.2 Estados de la cuenta . . . . .                                  | 21        |
| 5.3 Conjuntos de roles . . . . .                                    | 21        |
| 5.4 Mecanismo de control de acceso . . . . .                        | 21        |
| Capa 1 - Decorador para vistas HTML . . . . .                       | 21        |
| Capa 2 - Permisos de clase para la API . . . . .                    | 22        |
| Capa 3 - Contexto de plantilla . . . . .                            | 22        |
| 5.5 Matriz de permisos por rol . . . . .                            | 23        |
| 5.6 Matriz de acceso por modulo . . . . .                           | 24        |
| 5.7 Sesion y autenticacion . . . . .                                | 24        |
| <b>6. Modelo de datos . . . . .</b>                                 | <b>26</b> |
| 6.1 Convenciones del modelo de datos . . . . .                      | 26        |
| 6.2 Mapa relacional . . . . .                                       | 26        |
| 6.3 Estructura fisica: Jardines, edificios y apartamentos . . . . . | 27        |
| Estados de un departamento . . . . .                                | 27        |
| 6.4 Personas: propietarios y residentes . . . . .                   | 28        |
| Tipo de relacion del residente . . . . .                            | 28        |
| 6.5 Dinero: cargos de mantenimiento y pagos . . . . .               | 28        |
| Conceptos de pago . . . . .                                         | 29        |
| Estados de pago y morosidad . . . . .                               | 29        |
| 6.6 Incidencias y su ciclo de vida . . . . .                        | 29        |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| Ciclo de estados . . . . .                                     | 30        |
| Tablas Satellite . . . . .                                     | 30        |
| 6.7 Ficha del residencial . . . . .                            | 30        |
| 6.8 Inventario de modelos por aplicacion . . . . .             | 31        |
| 6.9 Decisiones de diseno del modelo . . . . .                  | 31        |
| PROTECT en los vinculos con el usuario que registra . . . . .  | 31        |
| SET_NULL donde la relacion es contextual . . . . .             | 32        |
| CASCADE donde la dependencia es fuerte . . . . .               | 32        |
| Restricciones de integridad a nivel de base de datos . . . . . | 32        |
| 6.10 Claves foraneas polimorficas manuales . . . . .           | 32        |
| <b>7. API REST . . . . .</b>                                   | <b>34</b> |
| 7.1 Configuracion global de DRF . . . . .                      | 34        |
| 7.2 Patron de los ViewSets . . . . .                           | 34        |
| 7.3 Inventario de endpoints . . . . .                          | 35        |
| 7.4 Operaciones CRUD generadas por el router . . . . .         | 36        |
| 7.5 Acciones personalizadas . . . . .                          | 36        |
| Autenticacion y perfil (accounts) . . . . .                    | 36        |
| Ciclo de vida de un visitante (visitors) . . . . .             | 36        |
| Reportes (reports) . . . . .                                   | 37        |
| 7.6 Filtros, busqueda y ordenamiento . . . . .                 | 37        |
| 7.7 Visibilidad en la API . . . . .                            | 37        |
| 7.8 Formato de las respuestas . . . . .                        | 37        |
| <b>8. Frontend: plantillas, CSS y JavaScript . . . . .</b>     | <b>39</b> |
| 8.1 Arquitectura del frontend . . . . .                        | 39        |
| 8.2 Plantillas . . . . .                                       | 39        |
| Plantilla base y componentes . . . . .                         | 39        |
| Plantillas por modulo . . . . .                                | 39        |
| 8.3 Sistema de diseno: variables.css . . . . .                 | 39        |
| Paleta de colores . . . . .                                    | 40        |
| Reglas de uso de las variables . . . . .                       | 40        |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| 8.4 Organizacion de los estilos . . . . .                           | 40        |
| 8.5 Convenciones de nombres en CSS . . . . .                        | 41        |
| 8.6 Reglas obligatorias del frontend . . . . .                      | 41        |
| 8.7 JavaScript . . . . .                                            | 42        |
| Organizacion . . . . .                                              | 42        |
| Como el JS recibe datos del HTML . . . . .                          | 42        |
| 8.8 Accesibilidad y experiencia de usuario . . . . .                | 43        |
| <b>9. Servidor ASGI, tiempo real y tareas asincronas . . . . .</b>  | <b>44</b> |
| 9.1 Modelo de despliegue previsto . . . . .                         | 44        |
| 9.2 Estado actual de ASGI . . . . .                                 | 44        |
| 9.3 Que falta para activar el tiempo real . . . . .                 | 44        |
| 9.4 Tareas asincronas (Celery) . . . . .                            | 45        |
| <b>10. Despliegue en produccion . . . . .</b>                       | <b>46</b> |
| 10.1 Requisitos de produccion . . . . .                             | 46        |
| 10.2 Checklist de despliegue . . . . .                              | 46        |
| 10.3 Conmutar a PostgreSQL . . . . .                                | 46        |
| 10.4 Seguridad en produccion . . . . .                              | 47        |
| 10.5 Lista de verificacion de seguridad antes de publicar . . . . . | 47        |
| <b>11. Problemas conocidos y deuda tecnica . . . . .</b>            | <b>49</b> |
| 11.1 Configuracion (settings.py) . . . . .                          | 49        |
| 11.2 URLs . . . . .                                                 | 49        |
| 11.3 Tiempo real . . . . .                                          | 49        |
| 11.4 Modelo de datos . . . . .                                      | 49        |
| 11.5 Seguridad y permisos . . . . .                                 | 50        |
| 11.6 Calidad y pruebas . . . . .                                    | 50        |
| 11.7 Orden de trabajo sugerido . . . . .                            | 51        |
| <b>12. Guia de mantenimiento y extension . . . . .</b>              | <b>52</b> |
| 12.1 Como agregar un modulo nuevo . . . . .                         | 52        |

12.2 Como agregar un campo a un modelo . . . . . 52

12.3 Buenas practicas al escribir codigo en este proyecto . . . . . 53

12.4 Como diagnosticar problemas . . . . . 53

12.5 Comandos de verificacion antes de publicar . . . . . 54

12.6 Resumen de la arquitectura en una linea . . . . . 54

# 1. Introduccion

## 1.1 Proposito del documento

Este manual describe el sistema **CONDYSOS** desde el punto de vista tecnico: que hace, como esta construido, que decisiones tecnicas lo sostienen y como instalarlo, operarlo y extenderlo. Todo el contenido se derivo directamente del codigo fuente del repositorio, de modo que el documento y el sistema evolucionan juntos.

El manual esta dirigido a las personas que desarrollaran, daran soporte o administrraran el sistema: perfil de desarrollo backend, perfil de desarrollo frontend y perfil de operador.

## 1.2 Que es CONDYSOS

CONDYSOS es un sistema web de administracion y gestion para residenciales verticales u horizontales. Centraliza la informacion de apartamentos, residentes, propietarios, cuotas, pagos, incidencias y control de accesos, reemplazando los cuadernos manuales y las hojas de calculo dispersas que normalmente se usan en la operacion de una administracion.

El sistema responde a un problema concreto: la administracion de un residencial maneja datos sensibles (personas, documentos, Plate PAGADO/saldo) y tareas recurrentes con plazos (cuotas, mantenimiento, permisos). Sin un sistema unico, cada tarea se resuelve por canales distintos y se pierde trazabilidad.

## 1.3 Alcance funcional

El alcance funcional se organiza en los siguientes modulos, cada uno desplegado como una aplicacion Django independiente pero integrada en un unico proyecto:

| Modulo                    | Funcion principal                                                                            | Usuarios principales                  |
|---------------------------|----------------------------------------------------------------------------------------------|---------------------------------------|
| <b>Inicio</b> (dashboard) | Resumen de estado: departamentos, incidencias abiertas, pagos por vencer, actividad reciente | Todos                                 |
| <b>Residencial</b>        | Ficha unica del residencial: nombre, RNC, direccion, provincia/municipio y distribucion      | Administrador                         |
| <b>Departamentos</b>      | Jerarquia Jardin → Edificio → Apartamento, con estado y propietario                          | Todos (lectura), Gestion (escritura)  |
| <b>Propietarios</b>       | Registro de propietarios y asociacion con apartamentos                                       | Gestion                               |
| <b>Residentes</b>         | Ocupantes por apartamento: relacion, fecha de ingreso, mascotas                              | Gestion, Propietario (lectura)        |
| <b>Pagos</b>              | Registro de cuotas y pagos, estados de mora, descarga de comprobante en PDF                  | Todos (lectura), Gestion (escritura)  |
| <b>Mantenimientos</b>     | Catalogo de cargos y cuotas recurrentes por periodicidad                                     | Todos (lectura), Gestion (escritura)  |
| <b>Incidencias</b>        | Reporte, clasificacion, asignacion, seguimiento e historial de cambios de estado             | Todos (reportar), Gestion (gestionar) |
| <b>Solicitudes</b>        | Certificados, permisos, instalaciones y tramites documentales                                | Gestion                               |
| <b>Visitantes</b>         | Registro, autorizacion y control de entrada/salida                                           | Seguridad, Gestion                    |
| <b>Areas comunes</b>      | Catalogo de areas comunes con tipo, capacidad, horario y estado                              | Todos (lectura), Gestion (escritura)  |

| Modulo         | Funcion principal                                                        | Usuarios principales                 |
|----------------|--------------------------------------------------------------------------|--------------------------------------|
| Reservas       | Solicitud y aprobacion de reservas de areas comunes                      | Todos                                |
| Comunicados    | Avisos, eventos y mantenimiento de comunicados del residencial           | Todos (lectura), Gestion (escritura) |
| Notificaciones | Avisos dirigidos a usuarios concretas                                    | Gestion                              |
| Reportes       | Reportes administrativos, ocupacion, pagos y bitacora de auditoria (CSV) | Gestion                              |
| Chat           | Mensajeria persistida entre usuarios y grupos                            | Todos                                |
| Cuentas        | Alta, edicion, activacion y roles de usuarios; perfil propio             | Administrador                        |

Tabla 1.1 - Modulos funcionales de CONDYSOS y su acceso por rol.

## 1.4 Perfiles de usuario

El sistema define cinco roles, almacenados en el campo `role` del modelo de usuario. Cada rol habilita un conjunto de modulos y un conjunto de acciones (crear, editar, eliminar):

- **Administrador** (`admin`): acceso total. Es el unico rol que autoriza el registro de nuevos usuarios y gestiona roles.
- **Encargado de Administracion** (`manager`): opera la operacion diaria (pagos, solicitudes, residentes, reportes) pero no administra usuarios ni estructura del sistema.
- **Residente** (`resident`): consulta sus datos, reporta incidencias y reserva areas comunes.
- **Propietario** (`propietario`): consulta informacion de los apartamentos que possui.
- **Seguridad / Porteria** (`security`): opera el control de visitantes y accesos; sin acceso financiero ni a datos de residentes.

### NOTA

**El detalle normativo de permisos por rol esta en el capitulo 5.**

La matriz completa (que puede y que no puede hacer cada rol) se documenta en AGENTS.md y se resume aqui en el capitulo 5.

## 1.5 Convenciones del documento

- Los nombres de archivo, clases y funciones aparecen en tipo `monoespaciado`.
- Los bloques de codigo son extractos literales del repositorio.
- Las cajas **ATENCION** señalan riesgo de operacion; las cajas **PENDIENTE / BUG** señalan defectos conocidos del codigo actual.
- Las rutas se expresan siempre relativas a la raiz del proyecto Django.

## 2. Stack tecnologico y dependencias

### 2.1 Lenguaje y runtime

| Elemento                | Valor                                    | Comentario                                          |
|-------------------------|------------------------------------------|-----------------------------------------------------|
| Lenguaje                | Python 3.13+                             | Entorno de desarrollo verificado con 3.13.6         |
| Entorno virtual         | venv/ (Windows: venv\Scripts\python.exe) | Nunca se instalan paquetes en el Python del sistema |
| Gestion de dependencias | pip + requirements.txt                   | El archivo no fija versiones a proposito            |

#### ATENCION

##### requirements.txt no fija versiones

El archivo de dependencias usa comparadores `>=` sin versiones exactas. `pip install -r requirements.txt` instala siempre la ultima version compatible.

Consecuencia: una instalacion nueva puede diferir de la que se probó. Para reproducibilidad conviene congelar versiones con `pip freeze` en el momento de una publicacion.

Actualizacion controlada: `pip install --upgrade --dry-run -r requirements.txt` para ver que cambios sin aplicar nada.

### 2.2 Dependencias del proyecto

El archivo `requirements.txt` declara los siguientes paquetes, agrupados por proposito:

| Grupo         | Paquete             | Version min. | Uso en CONDYSOS                                         |
|---------------|---------------------|--------------|---------------------------------------------------------|
| Core          | django              | 6.1.1        | Framework base: ORM, vistas, plantillas, admin, auth    |
| API           | djangorestframework | 3.18.1       | ViewSet, serializers, permisos y paginacion de la API   |
| API           | django-filter       | 26.1         | DjangoFilterBackend para filtrar y ordenar listados     |
| Seguridad web | django-cors-headers | 4.9.0        | Middleware CORS para consumos desde otros origenes      |
| Datos         | psycopg             | 3.3.6        | Driver PostgreSQL para el despliegue de produccion      |
| Real-time     | channels            | 4.3.2        | Infraestructura ASGI para WebSockets                    |
| Real-time     | channels-redis      | 4.3.0        | Channel layer distribuido sobre Redis                   |
| Real-time     | daphne              | 4.2.3        | Servidor ASGI; es el que ejecuta runserver              |
| Utilerias     | python-dotenv       | 1.2.3        | Carga del archivo .env en settings                      |
| Utilerias     | celery              | 5.6.3        | Cola de tareas asincronas (configurado, sin uso activo) |
| Utilerias     | redis               | 8.1.0        | Cliente Redis                                           |
| Utilerias     | Pillow              | 12.3.0       | Procesamiento de imagenes (avatars, fotos)              |
| Documentos    | reportlab           | 5.0.1        | Generacion del comprobante de pago en PDF               |
| Produccion    | gunicorn            | 26.2.0       | Servidor WSGI para produccion                           |
| Produccion    | whitenoise          | 6.12.0       | Servido de archivos estaticos sin servidor extra        |

| Grupo      | Paquete              | Version min. | Uso en CONDYSOS                   |
|------------|----------------------|--------------|-----------------------------------|
| Desarrollo | django-extensions    | 4.1          | Comandos de gestion adicionales   |
| Desarrollo | django-debug-toolbar | 8.0.0        | Panel de depuracion (no activado) |

Tabla 2.1 - Dependencias declaradas en requirements.txt.

## 2.3 Componentes principales

| Componente        | Tecnologia                        | Responsabilidad en el sistema                              |
|-------------------|-----------------------------------|------------------------------------------------------------|
| Backend web       | Django 6.1                        | MVT: URLs, vistas, plantillas, ORM y administracion        |
| API REST          | Django REST Framework             | ViewSets por modulo con permisos, filtros y paginacion     |
| Base de datos     | SQLite (dev) / PostgreSQL (prod)  | Persistencia relacional de 23 modelos                      |
| Plantillas        | Django Template Language          | HTML renderizado en el servidor                            |
| Estilos           | CSS con variables + media queries | Sistema de diseno compartido entre modulos                 |
| Interactividad    | JavaScript nativo (sin framework) | Formularios, dialogos, filtros y acciones asincronas       |
| PDF               | ReportLab                         | Comprobante de pago descargable                            |
| Tiempo real       | Daphne + Channels + Redis         | WebSockets de chat y notificaciones (pendiente de cablear) |
| Tareas asincronas | Celery + Redis                    | Configurado en settings, sin app Celery implementada       |
| Despliegue        | Gunicorn + WhiteNoise             | Servidor de produccion previsto                            |

Tabla 2.2 - Componentes tecnologicos y su papel en la arquitectura.

## 2.4 Dependencias declaradas pero no activas

Los siguientes paquetes aparecen en requirements.txt y/o en settings.py pero no estan integrados en INSTALLED\_APPS ni en el flujo de ejecucion. No son errores, pero conviene saberlo antes de planificar trabajo sobre ellos:

| Paquete / config               | Estado real                                     | Que faltaria                                                          |
|--------------------------------|-------------------------------------------------|-----------------------------------------------------------------------|
| channels                       | No esta en INSTALLED_APPS                       | Registrar la app y cablear el ProtocolTypeRouter en condosys/asgi.py  |
| chat/routing.py + consumers.py | Existe pero es codigo muerto                    | Registrar las rutas WebSocket en asgi.py con AuthMiddlewareStack      |
| CHANNEL_LAYERS                 | Configurado con RedisChannelLayer               | Solo se usa si se cablea el routing ASGI                              |
| celery                         | Configurado (broker, serializers, timezone)     | Crear celery.py, el initializer de la app y registrar tareas          |
| django-extensions              | No esta en INSTALLED_APPS                       | Anadir la app para habilitar sus comandos                             |
| django-debug-toolbar           | No esta en INSTALLED_APPS                       | Anadir app + middleware solo en desarrollo                            |
| corsheaders                    | Middleware activo pero la app no esta instalada | Anadir 'corsheaders' a INSTALLED_APPS si se consume desde otro origen |
| gunicorn + whitenoise          | Solo dependencias                               | Crear el servicio de produccion y servir los estaticos con WhiteNoise |
| psycopg                        | Driver instalado                                | Descomentar el bloque PostgreSQL de settings.py                       |

Tabla 2.3 - Capacidades preparadas pero no habilitadas.

**ATENCION****El frontend no usa ningun framework de JavaScript**

Todo el comportamiento dinamico esta escrito en JavaScript nativo spread en 29 archivos .js. No hay build step, ni bundler, ni node\_modules, ni transpilador.

Ventaja: no hay cadena de herramientas que mantener.

Riesgo: el JavaScript del proyecto crece de forma lineal con los modulos y no tiene pruebas ni linting automatico.

## 3. Arquitectura del proyecto

### 3.1 Vision general

CONDYSYS sigue el patron clasico de Django: un unico proyecto de settings y URLs que agrega multiples aplicaciones. Cada aplicacion es autonomous en cuanto a modelos, vistas, serializers, plantillas y estaticos, pero comparte el proyecto, la base de datos, el sistema de autenticacion y los permisos definidos en la aplicacion accounts.

#### *Flujo de una peticion en CONDYSOS*

```

Navegador
|
[ Daphne (ASGI) ] daphne esta primero en INSTALLED_APPS
|
+-----+-----+
| |
[Vistas HTML] [API REST (DRF)]
| |
+-----+-----+
|
[ Capa comun ]
accounts.permissions (IsAdmin, IsManager,
IsResident, IsGestionOrSoloLectura, ...)
accounts.decorators (@role_required)
|
[ ORM Django ]
|
[ SQLite (dev) / PostgreSQL (prod) ]

Contexto transversal: Jinja-free Django Templates,
static/css (variables.css como fuente unica de tokens),
static/js (modulos sin framework).
```

### 3.2 Estructura de directorios

El repositorio tiene dos niveles: el proyecto Django y, dentro de el, las aplicaciones. La raiz del proyecto es el directorio que contiene manage.py.

#### *Estructura del repositorio*

```

condosys/
|-- manage.py # utilidad de linea de comandos de Django
|-- requirements.txt # dependencias del proyecto
|-- .gitignore # protege .env, db.sqlite3, .venv, __pycache__
|-- venv/ # entorno virtual (no versionado)
|-- db.sqlite3 # base de datos de desarrollo (no versionada)
|
|-- condosys/ # CONFIGURACION del proyecto
| |-- settings.py # settings unicos (contiene duplicados, ver cap. 11)
| |-- urls.py # ROOT_URLCONF: incluye los urls de cada app
| |-- asgi.py # punto de entrada ASGI (sin ProtocolTypeRouter)
| |-- wsgi.py # punto de entrada WSGI
| `-- accounts/ # sub-paquete auxiliar interno
|
|-- templates/ # plantillas GLOBALES
| |-- base.html # esqueleto comun a toda la interfaz
| `-- componentes/ # header, aside, modal, paginacion, botones, ...
|
|-- static/ # estaticos COMPARTIDOS del proyecto
| |-- css/
| | |-- variables.css # FUENTE UNICA de tokens de diseno
| | |-- style.css # estilos base (importa variables.css)
| | |-- formularios.css # formularios compartidos
| | |-- tablas.css # tablas de datos
| | |-- paginacion.css # controles de paginacion
| | |-- dialogos.css # modales y dialogos
| | |-- acciones.css # paneles de acciones del dashboard
| | |-- entradas.css # inputs
| | `-- mediaquerys.css # breakpoints comunes
| |-- js/
| | |-- dialogos.js # gestion de <dialog>
| | |-- pagos.js # logica compartida de pagos
| | `-- mascaras.js, contadores.js, menu-movil.js, ...
| `-- images/icons/ # iconografia
|
|-- <app>/ # 18 aplicaciones Django
|-- models.py # modelos de datos
|-- views.py # vistas HTML + ViewSets DRF
|-- serializers.py # serializers de la API
|-- forms.py # formularios de las vistas HTML
|-- urls.py # rutas + DefaultRouter
|-- admin.py # registro en el admin de Django
|-- tests.py # plantilla vacia (no hay suite real)
|-- migrations/ # migraciones de esquema
|-- templates/<app>/ # plantillas de la app
`-- static/css, static/js # estilos y scripts especificos del modulo

```

### 3.3 El patron por aplicacion

Todas las apps de dominio siguen el mismo esqueleto. Cuando agregue un modulo nuevo, copie este patron:

| Archivo        | Contenido esperado                                           | Existe en                  |
|----------------|--------------------------------------------------------------|----------------------------|
| models.py      | Modelos de dominio con claves UUID y related_name explicitos | Todas salvo login e inicio |
| views.py       | app_index() con @role_required + ViewSets DRF                | Todas                      |
| serializers.py | Serializers de lectura (y de detalle cuando aplica)          | Todas salvo inicio         |

| Archivo        | Contenido esperado                                  | Existe en                    |
|----------------|-----------------------------------------------------|------------------------------|
| urls.py        | Rutas HTML + DefaultRouter registrado en su prefijo | Todas                        |
| forms.py       | Formularios ModelForm para las vistas HTML          | Mayoria                      |
| admin.py       | Registro de modelos en el admin de Django           | Todas                        |
| permissions.py | Permisos DRF especificos                            | Solo accounts (ver cap. 5)   |
| migrations/    | Migraciones de esquema                              | Todas salvo inicio y login   |
| templates/     | index.html como pagina principal del modulo         | Todas                        |
| static/        | CSS y JS propios del modulo                         | Todas salvo inicio (parcial) |
| tests.py       | Plantilla vacia generada por startapp               | Todas                        |

Tabla 3.1 - Archivos esperados en cada aplicacion de dominio.

#### NOTA

##### Convencion de nombres del frontend

La pagina principal de cada modulo se llama siempre index.html; los demas archivos se nombran por su contenido (agregar.html, nuevo.html, perfil.html).

Los estilos y scripts compartidos viven en static/css y static/js del proyecto; lo especifico de un modulo queda en el static de su propia app.

## 3.4 Capa de presentacion: dos superficies en la misma app

Cada modulo expone dos superficies complementarias:

- **Superficie HTML** - vista de funcion `app_index` que renderiza la plantilla `index.html` con paginacion server-side, filtros por query string y formularios. Es la experiencia principal del usuario.
- **Superficie API** - `ViewSet` de DRF registrado en el router que expone listado, detalle, creacion, actualizacion y borrado en JSON, con filtros, busqueda, ordenamiento y paginacion por `?page=`.

La vista HTML aplica el permiso con el decorador `@role_required(...)`; la API lo aplica con `permission_classes`. Los listados HTML tiemblen paginacion de 15 elementos por pagina; la API usa `PageNumberPagination` con 20.

## 3.5 Patrones de implementacion recurrentes

### Visibilidad por usuario en los listados

Los listados HTML aplican un filtro de visibilidad para que un residente solo vea lo que le corresponde. El patron es una funcion helper `_<entidad>visibles(user)` en `views.py`, reutilizada por la vista y por el `get_queryset()` del `ViewSet`:

#### Filtro de visibilidad por usuario

```
# Patron real (payments/views.py, estructura logica)
def _pagos_visibles(user):
    qs = Payment.objects.all()
    if user.role in ('admin', 'manager'):
        return qs
    # El residente ve los pagos de sus apartamentos y sus propios registros
    return qs.filter(Q(resident__user=user) | Q(apartment__residents__user=user)) \
        .distinct()

# El ViewSet lo reutiliza para no duplicar la regla
def get_queryset(self):
    qs = _pagos_visibles(self.request.user)
    ... # filtros opcionales por query string
    return qs.distinct()
```

### Serializers diferenciados para listado y detalle

Cuando el detalle expone mas campos que el listado, el ViewSet elige el serializer segun la accion mediante `get_serializer_class()`:

```
def get_serializer_class(self):
    if self.action == 'retrieve':
        return ApartmentDetailSerializer # incluye datos anidados del edificio
        return ApartmentListSerializer # vista compacta para listados

# Uso en accounts: el serializer de creacion y el de actualizacion
# exigen campos distintos, asi que tambien se selecciona por accion.
def get_serializer_class(self):
    if self.action == 'create':
        return UserCreateSerializer # incluye password (write_only)
    if self.action in ('update', 'partial_update'):
        return UserUpdateSerializer # no expone role ni status
    return UserSerializer
```

### Cambios de estado auditados

El modulo de incidencias registra cada transicion de estado en una bitacora de historial. Se hace sobrescribiendo `perform_update()` en el ViewSet:

#### *incidents/views.py - perform\_update*

```
def perform_update(self, serializer):
    estado_previo = serializer.instance.status
    instancia = serializer.save()

    # Si cambian el estado o la asignacion, deja rastro en el historial
    if (estado_previo != instancia.status) or \
        (instancia.assigned_to_id != self._asignacion_previa()):
        IncidentHistory.objects.create(
            incident=instancia,
            status_from=estado_previo,
            status_to=instancia.status,
            comment=self.request.data.get('history_comment', ''),
            changed_by=self.request.user,
        )
```

### Comprobante en PDF

El comprobante de pago se genera en el servidor con ReportLab y se devuelve como respuesta HTTP en lugar de renderizar una plantilla:

***payments/views.py - generar\_comprobante***

```
buffer = BytesIO()
doc = SimpleDocTemplate(buffer, pagesize=letter, ...)
doc.build([...]) # elementos del PDF
buffer.seek(0)

respuesta = HttpResponse(buffer.getvalue(), content_type='application/pdf')
respuesta['Content-Disposition'] = 'inline; filename="comprobante_pago.pdf"'
return respuesta
```

Los parametros viajan por query string (?pago\_id=...&departamento=...) porque el JS externo no puede usar etiquetas de plantilla Django; los valores viajan en atributos data-\* del HTML.

## 4. Instalacion, ejecucion y operacion

### 4.1 Requisitos previos

| Requisito             | Detalle                               | Como verificarlo        |
|-----------------------|---------------------------------------|-------------------------|
| Python                | 3.13 o superior (probado en 3.13.6)   | python --version        |
| pip                   | incluido con Python                   | python -m pip --version |
| Acceso al repositorio | git clone del remoto origin           | git remote -v           |
| Redis (opcional)      | Solo si se activan Channels o Celery  | redis-cli ping          |
| PostgreSQL (opcional) | Solo para el despliegue de produccion | psql --version          |

### 4.2 Instalacion desde cero

Todos los comandos se ejecutan desde la raiz del proyecto, el directorio que contiene `manage.py`. En Windows, los comandos usan la ruta relativa del interprete del entorno virtual.

#### Paso 1 - Clonar el repositorio

```
git clone https://github.com/JMJ0331/condosys.git
cd condosys
git checkout desarrollo # rama de trabajo; main es la estable
```

#### Paso 2 - Crear el entorno virtual

```
# Windows (PowerShell / CMD)
python -m venv venv
venv\Scripts\activate

# Linux / macOS
python3 -m venv venv
source venv/bin/activate

# Verificacion
python --version # debe responder 3.13.x
```

A partir de aqui, este manual usa la forma Windows `venv\Scripts\python.exe` porque es el entorno verificado. En Linux/macOS se sustituye por `python`.

#### Paso 3 - Instalar dependencias

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# Verificacion: la suite de comprobacion debe reportar 0 problemas
python manage.py check
```

#### Paso 4 - Configurar el archivo .env

El proyecto lee su configuracion sensible desde un archivo `.env` en la raiz, cargado por `python-dotenv`. El archivo esta en `.gitignore` y nunca debe versionarse.

### ***.env - variables disponibles***

```
# .env (copiar de .env.example si existe; si no, crearlo a mano)
DJANGO_DEBUG=True
DJANGO_SECRET_KEY=<clave-larga-aleatoria>
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1

# Opcionales: solo si el despliegue los requiere
CORS_ALLOWED_ORIGINS=https://app.ejemplo.do
CSRF_TRUSTED_ORIGINS=https://app.ejemplo.do
REDIS_HOST=localhost
REDIS_PORT=6379
DATABASE_ENGINE=django.db.backends.postgresql
DATABASE_NAME=condosys_db
DATABASE_USER=postgres
DATABASE_PASSWORD=<clave>
DATABASE_HOST=localhost
DATABASE_PORT=5432
```

### **PENDIENTE / BUG**

#### **SECRET\_KEY no tiene valor por defecto**

`settings.py` lee `SECRET_KEY = os.getenv('DJANGO_SECRET_KEY')` sin fallback. Si la variable falta, Django lanza `ImproperlyConfigured` y el proyecto no arranca.

Implica que `.env` es obligatorio: no se puede levantar el proyecto sin el.

## **Paso 5 - Aplicar migraciones**

```
python manage.py makemigrations # solo si modificaste modelos
python manage.py migrate
```

## **Paso 6 - Cargar datos de ejemplo (opcional)**

El proyecto incluye un comando `deManagement` idempotente que siembra registros de prueba en todas las secciones. Es la via recomendada para levantar un entorno de demostracion o para verificar permisos con los distintos roles.

### ***Carga de datos de demostracion***

```
python manage.py poblar_datos

# Crea 7 usuarios base; las contraseñas de los recién creados son: Condosys123*
# admin@condosys.do rol admin
# gestion@condosys.do rol manager
# residente1..5@condosys.do rol resident
# Ejecutarlo de nuevo no duplica registros (update_or_create + transaction.atomic).
```

### **ATENCION**

#### **No use datos de ejemplo en produccion**

El comando `poblar_datos` crea usuarios con contraseña conocida y datos personales ficticios. Eliminelos antes de exponer el sistema a usuarios reales.

## Paso 7 - Arrancar el servidor de desarrollo

```
python manage.py runserver
# http://127.0.0.1:8000/
```

Aunque el comando se llama runserver, el servidor real es Daphne: la cadena daphne aparece primera en INSTALLED\_APPS y Django delega en ella cuando esta disponible.

## 4.3 Comandos de uso frecuente

| Comando                        | Para que sirve                                          |
|--------------------------------|---------------------------------------------------------|
| manage.py runserver            | Levanta el servidor ASGI de desarrollo                  |
| manage.py check                | Verifica la configuracion; debe dar 0 problemas         |
| manage.py migrate              | Aplica las migraciones pendientes                       |
| manage.py makemigrations       | Genera migraciones tras cambiar modelos                 |
| manage.py sqlmigrate <app> <n> | Muestra el SQL de una migracion                         |
| manage.py showmigrations       | Lista el estado de las migraciones de cada app          |
| manage.py createsuperuser      | Crea el administrador inicial (pide email, no username) |
| manage.py shell                | Consola interactiva de Python con el ORM cargado        |
| manage.py dbshell              | Consola directa de la base de datos                     |
| manage.py collectstatic        | Reúne los estaticos en STATIC_ROOT para produccion      |
| manage.py test                 | Ejecuta la suite de pruebas (hoy vacia)                 |
| manage.py poblar_datos         | Carga datos de demostracion (idempotente)               |

Tabla 4.1 - Comandos de gestion de Django usados en el proyecto.

## 4.4 Flujo de trabajo con git

| Concepto           | Convencion del proyecto                                                                                 |
|--------------------|---------------------------------------------------------------------------------------------------------|
| Rama de trabajo    | <b>desarrollo</b> - aqui se desarrolla                                                                  |
| Rama estable       | <b>main</b> - versiones estables                                                                        |
| Remoto             | origin -> <a href="https://github.com/JMJ0331/condosys.git">https://github.com/JMJ0331/condosys.git</a> |
| Prefijos de commit | add:, fix:, update:, refactor:                                                                          |
| Tiempo verbal      | Pretérito indefinido: 'agregué', 'integré', 'arreglé'                                                   |
| Idioma             | Español, incluidos los mensajes de commit                                                               |
| Ejemplo            | fix: corriji la validación del rol en el login                                                          |

```
git status # revisar antes de commitear
git add <archivos-especificos> # nunca commitear .env ni db.sqlite3
git commit -m "add: agregué módulo de reservas"
git push origin desarrollo
```

## 4.5 Rutas principales del sistema

| Ruta                | Modulo                             | Acceso                  |
|---------------------|------------------------------------|-------------------------|
| /                   | Inicio de sesion                   | Publico                 |
| /salir/             | Cierre de sesion (POST)            | Autenticado             |
| /inicio/            | Dashboard                          | Todos                   |
| /mi-perfil/         | Perfil del usuario                 | Todos                   |
| /residencial/       | Ficha del residencial              | Administrador           |
| /cuentas/           | Usuarios y roles                   | Administrador           |
| /departamentos/     | Jardines, edificios y apartamentos | Todos                   |
| /propietarios/      | Propietarios                       | Gestion                 |
| /residentes/        | Residentes y ocupantes             | Gestion, Propietario    |
| /pagos/             | Pagos y cuotas                     | Todos (Gestion escribe) |
| /pagos/comprobante/ | Comprobante en PDF                 | Todos                   |
| /mantenimientos/    | Cargos de mantenimiento            | Todos (Gestion escribe) |
| /incidencias/       | Incidencias                        | Todos                   |
| /solicitudes/       | Solicitudes y tramites             | Gestion                 |
| /visitantes/        | Control de visitantes              | Seguridad, Gestion      |
| /areas-comunes/     | Areas comunes                      | Todos                   |
| /reservas/          | Reservas                           | Todos                   |
| /comunicados/       | Comunicados                        | Todos                   |
| /notificaciones/    | Notificaciones                     | Gestion                 |
| /reportes/          | Reportes y bitacora                | Gestion                 |
| /chat/              | Mensajeria                         | Todos                   |
| /admin/             | Administracion de Django           | Staff                   |

Tabla 4.2 - Rutas HTML por modulo.

## 5. Usuarios, roles y permisos

### 5.1 Modelo de usuario

El sistema no usa el modelo de usuario por defecto de Django. `accounts.User` extiende `AbstractUser` con dos cambios de fondo:

| Cambio               | Detalle                                                          | Consecuencia                                                        |
|----------------------|------------------------------------------------------------------|---------------------------------------------------------------------|
| Clave primaria UUID  | <code>id = UUIDField(default=uuid.uuid4)</code>                  | Los identificadores no son enumerables; se pueden exponer en la API |
| Sin username         | <code>username = None; USERNAME_FIELD = 'email'</code>           | El login, el admin y <code>createsuperuser</code> piden email       |
| Campo role           | <code>admin   manager   resident   propietario   security</code> | El permiso se decide por rol, no por grupos                         |
| Campo status         | <code>active   inactive   pending</code>                         | El login manual rechaza a quien no tenga 'active'                   |
| Telefono y documento | <code>phone, document (unicos)</code>                            | Identificacion operativa del residente                              |
| Avatar               | <code>avatar (ImageField) y avatar_url (URLField)</code>         | Permite imagen local o remota                                       |
| Vinculo a jardin     | <code>garden = FK a structure.Garden (SET_NULL)</code>           | Encuadre del usuario dentro del residencial                         |

Tabla 5.1 - Diferencias del modelo de usuario frente al de Django.

### 5.2 Estados de la cuenta

| Estado   | Significado                                | Puede iniciar sesion |
|----------|--------------------------------------------|----------------------|
| active   | Cuenta habilitada                          | Si                   |
| pending  | En verificacion, pendiente de autorizacion | No                   |
| inactive | Cuenta desactivada                         | No                   |

El campo `is_active` (heredado de Django) convive con `status`. La vista de login exige las dos condiciones: `user.is_active and user.status == 'active'`. Mantenga ambas sincronizadas.

### 5.3 Conjuntos de roles

```
# accounts/permissions.py
ROLES_ADMIN = ('admin',)
ROLES_GESTION = ('admin', 'manager')
ROLES_SEGURIDAD = ('admin', 'manager', 'security')
ROLES_RESIDENTE = ('admin', 'manager', 'resident', 'propietario')
ROLES_TODO = ('admin', 'manager', 'security', 'resident', 'propietario')
```

Usar siempre estas constantes en vez de escribir las tuplas a mano: si un rol se agrega despues, basta con actualizar la constante.

### 5.4 Mecanismo de control de acceso

El sistema aplica permisos en tres capas. Todas se apoyan en el campo `role`.

#### Capa 1 - Decorador para vistas HTML

***accounts/decorators.py***

```

from accounts.decorators import role_required
from accounts.permissions import ROLES_GESTION

@login_required
@role_required(*ROLES_GESTION)
def agregar_pago(request):
    ... # renderiza el formulario

```

Si el usuario no tiene el rol, el decorador muestra un mensaje de error y redirige: a `visitantes_index` para seguridad (su unico modulo) y a inicio en cualquier otro caso.

**Capa 2 - Permisos de clase para la API**

| Clase                  | Regla                                                      | Se aplica en                                                                 |
|------------------------|------------------------------------------------------------|------------------------------------------------------------------------------|
| IsAdmin                | role in ROLES_ADMIN                                        | UserViewSet (listar, crear), ResidencialViewSet                              |
| IsManager              | role in ROLES_GESTION                                      | Garden, Building, Propietario, Solicitud, AuditLog, Report                   |
| IsResident             | role in ROLES_RESIDENTE                                    | CommonAreaViewSet, ReservationViewSet                                        |
| IsGestionOrSoloLectura | Autenticado y (metodo seguro o role in ROLES_GESTION)      | Resident, Payment, MaintenanceCharge, AreaComun, Communication, Notification |
| CanModifyUser          | admin, o el propio usuario                                 | UserViewSet (detalle, editar, borrar)                                        |
| CanModifyIncident      | Gestion; o el que reporta; o el asignado (solo lectura)    | IncidentViewSet                                                              |
| CanModifyVisitor       | Gestion; o security; o residente del apartamento           | VisitorViewSet                                                               |
| CanAccessApartment     | Gestion; o propietario del apartamento; o residente actual | ApartmentViewSet                                                             |

Tabla 5.2 - Permisos DRF personalizados definidos en `accounts/permissions.py`.

**Capa 3 - Contexto de plantilla**

Los context processors `modulos_usuario` y `permisos_modulo` publican en el HTML los modulos visibles y los permisos de crear, editar y eliminar de cada modulo. Es lo que permite ocultar botones segun el rol sin duplicar logica en el template.

***Nombres de los context processors***

```
# accounts/context_processors.py - idea central
MODULOS = [
{'nombre': 'pagos', 'url': 'pagos_index', 'activo_en': [...]},
{'nombre': 'incidencias', 'url': 'incidencias_index', 'activo_en': [...]},
... # 14 modulos en total
]
ROL_CREADOR, ROL_EDITOR, ROL_ELIMINADOR = ... # tabla de roles por modulo

# Salida al template
{
'modulos_habilitados': [...], # que ver en el menu
'modulos_acceso': [...],
'puede_crear': True,
'puede_editar': True,
'puede_eliminar': False,
'permisos_modulos': {...},
}
```

## 5.5 Matriz de permisos por rol

La tabla siguiente resume el alcance funcional de cada rol. Es la referencia para responder preguntas de soporte y para validar requisitos antes de implementar.

| Capacidad                             | Admin | Gestion | Seguridad | Residente      | Propietario    |
|---------------------------------------|-------|---------|-----------|----------------|----------------|
| Autorizar registro de usuarios        | Si    | No      | No        | No             | No             |
| Gestionar roles y permisos            | Si    | No      | No        | No             | No             |
| Configurar ficha del residencial      | Si    | No      | No        | No             | No             |
| Registrar y editar departamentos      | Si    | Si      | No        | No             | No             |
| Registrar propietarios                | Si    | Si      | No        | No             | No             |
| Registrar residentes                  | Si    | Si      | No        | No             | No             |
| Registrar pagos y cuotas              | Si    | Si      | No        | No             | No             |
| Emitir comprobantes de pago           | Si    | Si      | No        | Si             | Si             |
| Consultar pagos                       | Si    | Si      | No        | Solo los suyos | Solo los suyos |
| Consultar pagos pendientes y vencidos | Si    | Si      | No        | No             | No             |
| Registrar cargos de mantenimiento     | Si    | Si      | No        | No             | No             |
| Reportar incidencias                  | Si    | Si      | No        | Si             | Si             |
| Cambiar estado de incidencias         | Si    | Si      | No        | No             | No             |
| Ver historial de incidencias          | Si    | Si      | No        | Si             | Si             |
| Gestionar solicitudes                 | Si    | Si      | No        | No             | No             |
| Registrar visitantes                  | Si    | Si      | Si        | No             | No             |
| Autorizar y controlar accesos         | Si    | Si      | Si        | No             | No             |
| Administrar areas comunes             | Si    | Si      | No        | No             | No             |
| Reservar areas comunes                | Si    | Si      | No        | Si             | Si             |

| Capacidad                     | Admin | Gestion | Seguridad | Residente | Propietario |
|-------------------------------|-------|---------|-----------|-----------|-------------|
| Publicar comunicados          | Si    | Si      | No        | No        | No          |
| Ver comunicados               | Si    | Si      | Si        | Si        | Si          |
| Ver reportes administrativos  | Si    | Si      | No        | No        | No          |
| Usar la bitacora de auditoria | Si    | Si      | No        | No        | No          |
| Acceder al panel /admin/      | Si    | No      | No        | No        | No          |

Tabla 5.3 - Alcance funcional por rol.

## 5.6 Matriz de acceso por modulo

Equivalente a la anterior, pero por modulo y por accion, que es como el codigo opera realmente.

| Modulo         | Ver                   | Crear / editar                  | Eliminar | Permiso DRF                    |
|----------------|-----------------------|---------------------------------|----------|--------------------------------|
| Inicio         | Todos                 | -                               | -        | role_required                  |
| Residencial    | Admin                 | Admin                           | Admin    | IsAdmin                        |
| Cuentas        | Admin                 | Admin                           | Admin    | IsAdmin / CanModifyUser        |
| Departamentos  | Todos                 | Gestion                         | Gestion  | IsManager / CanAccessApartment |
| Propietarios   | Gestion               | Gestion                         | Gestion  | IsManager                      |
| Residentes     | Gestion + propietario | Gestion                         | Gestion  | IsGestionOrSoloLectura         |
| Pagos          | Todos                 | Gestion                         | Gestion  | IsGestionOrSoloLectura         |
| Mantenimientos | Todos                 | Gestion                         | Gestion  | IsGestionOrSoloLectura         |
| Incidencias    | Todos                 | Todos (crear); Gestion (editar) | Gestion  | CanModifyIncident              |
| Solicitudes    | Gestion               | Gestion                         | Gestion  | IsManager                      |
| Visitantes     | Todos                 | Seguridad + Gestion             | Gestion  | CanModifyVisitor               |
| Areas comunes  | Todos                 | Gestion                         | Gestion  | IsGestionOrSoloLectura         |
| Reservas       | Todos                 | Todos                           | Gestion  | IsResident                     |
| Comunicados    | Todos                 | Gestion                         | Gestion  | IsGestionOrSoloLectura         |
| Notificaciones | Gestion               | Gestion                         | Gestion  | IsGestionOrSoloLectura         |
| Reportes       | Gestion               | Gestion                         | -        | IsManager                      |
| Chat           | Todos                 | Todos                           | Todos    | IsAuthenticated                |

Tabla 5.4 - Acceso por modulo y accion.

## 5.7 Sesion y autenticacion

- **Login por email.** settings.py define LOGIN\_URL = '/', LOGIN\_REDIRECT\_URL = '/inicio' y LOGOUT\_REDIRECT\_URL = '/'.
- **Redireccion por rol tras entrar.** Seguridad aterriza en visitantes\_index; admin y gestor sin residencial configurado, en residencial\_index; el resto, en inicio.

- **Sesion en base de datos** con duracion de 24 horas, expiracion al cerrar el navegador, cookie HttpOnly y SameSite=Lax.
- **CSRF** con cookie HttpOnly y origenes de confianza configurables.
- **API por sesion:** SessionAuthentication es la unica clase de autenticacion global, y el permiso por defecto es IsAuthenticated.
- **Validacion de contraseñas:** longitud minima de 8, similitud con los datos del usuario, contraseñas comunes y sin solo digitos.

**ATENCION****El logout exige POST**

cerrar\_sesion solo ejecuta django\_logout cuando el metodo es POST. Enlazarlo con un enlace <a href> no cierra la sesion; use un formulario con boton o JavaScript.

## 6. Modelo de datos

---

### 6.1 Convenciones del modelo de datos

- **Clave primaria UUID** en casi todos los modelos (`id = UUIDField(default=uuid.uuid4, editable=False)`). Excepcion: `chat.ChatGroup`, `incidents.IncidentImage` y `reports.AuditLogDetail` usan `AutoField` implicito.
- **related\_name explicito** en todas las relaciones inversas. No hay relaciones de tipo `ManyToMany` propias ni `OneToOne` en todo el proyecto.
- **on\_delete explicito** en cada `ForeignKey`. Se usan los cuatro valores con significado distinto: `CASCADE`, `PROTECT`, `SET_NULL` y (en un caso) `SET_DEFAULT`.
- **Indices declarados** en los campos por los que se filtra o se agrupa (estado, periodo, fechas), y en los compuestos de las consultas frecuentes.
- **unique\_together** en las entidades que pertenecen a un padre: un edificio no repite nombre dentro de un jardin, un apartamento no repite nombre dentro de un edificio.
- **Sin logica de negocio en save()** salvo el caso unico de `residencial.Residencial`, que documenta su excepcion (ver 6.7).

### 6.2 Mapa relacional

*Diagrama de relaciones (resumido)*

```

accounts.User --SET_NULL--> structure.Garden (users)
|
-- es referenciado por casi todos los modulos

structure.Garden --CASCADE--> structure.Building --CASCADE--> structure.Apartment
|
+-----+
| SET_NULL (owner) CASCADE PROTECT |
v v v v
propietarios.residents.Resident payments.Payment structure.Apartment
.Propietario | | |
| CASCADE | CASCADE | registered_by | SET_NULL
v v \--SET_NULL--> accounts.User
accounts.User structure.Apartment

incidents.Incident --CASCADE--> Apartment
+---CASCADE/SET_NULL--> Resident
+---PROTECT--> User (reported_by)
+---SET_NULL--> User (assigned_to)
+--> IncidentImage (CASCADE, related_name='images')
\--> IncidentHistory (CASCADE, related_name='history',
changed_by PROTECT sin related_name)

solicitudes.Solicitud --CASCADE--> Apartment; SET_NULL--> Resident
visitors.Visitor --CASCADE--> Apartment; PROTECT--> User (registered_by);
SET_NULL--> User (authorized_by)
maintenance.MaintenanceCharge --SET_NULL--> Apartment (NULL = aplica a todos)
communications.Communication --CASCADE--> Garden; PROTECT--> User (sender)
reservations.Reservation --CASCADE--> areas_comunes.AreaComun
SET_NULL--> Apartment / Resident
CASCADE/SET_NULL--> User (reserved_by / approved_by)
notifications.Notification --CASCADE--> User
chat.ChatMessage --CASCADE--> User (sender / receiver);
SET_NULL--> chat.ChatGroup
reports.AuditLog --CASCADE--> User
\--> AuditLogDetail (CASCADE, related_name='details')

residencial.Residencial <- modelo aislado, sin ninguna FK

```

## 6.3 Estructura fisica: Jardines, edificios y apartamentos

El nucleo espacial del sistema es una jerarquia de tres niveles que determina la organizacion de casi todos los demas modulos: los pagos cuelgan de un apartamento, los residentes de un apartamento, las incidencias de un apartamento.

| Modelo    | App       | Campos principales                                           | Relaciones                                          |
|-----------|-----------|--------------------------------------------------------------|-----------------------------------------------------|
| Garden    | structure | name (unico), location, description, is_active               | building_set, users, common_areas, communications   |
| Building  | structure | name, tower, block, number_of_floors, description, is_active | garden (CASCADE, related_name='buildings')          |
| Apartment | structure | name, floor, status, photo, is_active                        | building (CASCADE), owner (SET_NULL -> Propietario) |

Tabla 6.1 - Jerarquia espacial.

### Estados de un departamento

| Valor       | Etiqueta      | Significado                        |
|-------------|---------------|------------------------------------|
| empty       | Vacio         | Disponible, sinmogador asignado    |
| occupied    | Ocupado       | Alquilado o en uso                 |
| maintenance | En reparacion | No disponible temporalmente        |
| blocked     | Bloqueado     | Inhabilitado por la administracion |

El estado por defecto de un apartamento nuevo es empty. El estado se filtra en la vista HTML mediante el parametro ?estado= y en la API mediante ?status=.

## 6.4 Personas: propietarios y residentes

El sistema distingue dos figuras juridicas distintas y las modela por separado:

| Aspecto                     | propietarios.Propietario        | residents.Resident                            |
|-----------------------------|---------------------------------|-----------------------------------------------|
| Que representa              | El dueño del inmueble           | La persona que ocupa el apartamento           |
| Clave de negocio            | cedula (unica)                  | cedula (unica)                                |
| Relacion con el apartamento | Apartment.owner (SET_NULL)      | Resident.apartment (CASCADE, obligatorio)     |
| Relacion con el usuario     | user (CASCADE, opcional)        | user (CASCADE, opcional)                      |
| Datos de contacto           | phone, email, emergency_contact | phone, email, emergency_contact               |
| Estado civil                | marital_status (5 opciones)     | marital_status (5 opciones, duplicadas)       |
| Datos adicionales           | photo                           | photo, tipo_relacion, fecha_ingreso, mascotas |
| Orden por defecto           | full_name                       | apartment, full_name                          |

Tabla 6.2 - Propietario frente a residente.

**Como leerlo:** un apartamento puede tener un propietario y N residentes. El propietario no necesita estar, registrado como usuario; el residente puede ser un ocupante sin cuenta. Ambos pueden vincularse a un accounts.User opcional para que pueda iniciar sesion y ver su informacion.

### Tipo de relacion del residente

| Valor       | Etiqueta    | Implicacion                            |
|-------------|-------------|----------------------------------------|
| propietario | Propietario | El residente es tambien el dueño       |
| inquilino   | Inquilino   | Ocupa en alquiler                      |
| familiar    | Familiar    | Pariente del titular                   |
| ocupante    | Ocupante    | Sin relacion contractual (por defecto) |

## 6.5 Dinero: cargos de mantenimiento y pagos

El modulo financiero separa el **concepto** de lo que se cobra del **hecho de pago**. Es la unica forma de tener un catalogo de tarifas independiente de los movimientos reales.

|        | maintenance.MaintenanceCharge | payments.Payment                |
|--------|-------------------------------|---------------------------------|
| Que es | El cargo o cuota a cobrar     | El pago efectivamente realizado |

|                    | maintenance.MaintenanceCharge                                 | payments.Payment                                                             |
|--------------------|---------------------------------------------------------------|------------------------------------------------------------------------------|
| Campos clave       | concept, periodicity, amount, effective_date, payment_methods | amount, concept, period, payment_date, payment_method, status, receipt_image |
| Periodicidad       | monthly, quarterly, semiannual, annual                        | No aplica (se registra por periodo)                                          |
| Apartamento        | Opcional. NULL significa 'aplica a todos'                     | Obligatorio (PROTECT)                                                        |
| Residente          | -                                                             | Obligatorio (PROTECT)                                                        |
| Estado             | is_active (activo/inactivo)                                   | pending, at_risk, overdue, paid, cancelled                                   |
| Orden por defecto  | -effective_date                                               | -period                                                                      |
| Validacion cruzada | -                                                             | clean(): el residente debe pertenecer al apartamento elegido                 |

Tabla 6.3 - Catalogo de cargos frente a registro de pagos.

## Conceptos de pago

| Valor         | Etiqueta             | Tipo de obligacion          |
|---------------|----------------------|-----------------------------|
| maintenance   | Mantenimiento        | Cuota ordinaria             |
| extraordinary | Cuota extraordinaria | Cargo adicional puntual     |
| reservation   | Reserva              | Cargo por uso de area comun |
| parking       | Parqueo              | Cargo fijo                  |
| services      | Servicios            | Consumo (agua, luz)         |
| other         | Otro                 | Cualquier otro concepto     |

## Estados de pago y morosidad

| Estado    | Etiqueta  | Significado operativo                                        |
|-----------|-----------|--------------------------------------------------------------|
| pending   | Pendiente | Registrado, aun no pagado, sin retraso                       |
| at_risk   | En riesgo | Proximo a vencer; aparece en el panel de pagos por vencer    |
| overdue   | Vencido   | Vencio su periodo sin pago; entra en el reporte de morosidad |
| paid      | Pagado    | Cancelado                                                    |
| cancelled | Anulado   | Registro invalidado, no cuenta para balances                 |

### NOTA

#### Como se modela el periodo

El campo period es un DateField que representa el PRIMER dia del mes que se cobra (por ejemplo 2026-09-01 para septiembre de 2026).

Los filtros de la vista HTML (mes, anio) y de la API (month, year) operar sobre ese campo; las funciones de los reportes calculan el rango de meses con esa misma convencion.

## 6.6 Incidencias y su ciclo de vida

El modulo de incidencias es el mas rico del sistema: maneja clasificacion, prioridad, asignacion, evidencia, notas de resolucioe historial.

## Ciclo de estados

### Flujo de trabajo de una incidencia

```
[ new ] -> [ assigned ] -> [ in_progress ] -> [ resolved ] -> [ closed ]
Nueva Asignada En progreso Resuelta Cerrada
```

```
[ rejected ] (Rechazada) - salida alternativa desde new/assigned
[ resolved ] -> [ in_progress ] (reapertura)
```

| Campo            | Opciones                                                                   | Uso                        |
|------------------|----------------------------------------------------------------------------|----------------------------|
| category         | plumbing, electricity, structural, cleaning, security, noise, water, other | Clasificacion del reporte  |
| priority         | low, normal, high, urgent                                                  | Prioridad de atencion      |
| status           | new, assigned, in_progress, resolved, closed, rejected                     | Estado del flujo           |
| evidence         | FileField (upload a incidentes/evidencia/)                                 | Archivo adjunto principal  |
| resolution_notes | TextField                                                                  | Descripcion de la solucion |
| resolved_at      | DateTimeField                                                              | Momento de la resolucioe   |

Tabla 6.4 - Campos de clasificacion y seguimiento de una incidencia.

## Tablas Satellite

| Modelo          | Para que sirve                                                | Relacion                                   |
|-----------------|---------------------------------------------------------------|--------------------------------------------|
| IncidentImage   | Adjuntar imagenes mediante URLs externas a la incidencia      | incident (CASCADE, related_name='images')  |
| IncidentHistory | Bitacora de cada transicion de estado, con autor y comentario | incident (CASCADE, related_name='history') |

### ATENCION

#### Dos mecanismos de adjuntos coexisten

Incident.evidence (FileField, subida de archivo) e IncidentImage (tabla de URLs) cumplen la misma funcion. Antes de implementar, decida cual usar y evite generar adjuntos en ambos.

## 6.7 Ficha del residencial

residencial.Residencial es un modelo conceptualmente singular: describe el residencial al que pertenece el sistema. No tiene ninguna relacion con el resto de modelos.

| Grupo de campos | Contenido                                                                   |
|-----------------|-----------------------------------------------------------------------------|
| Identificacion  | nombre, rnc                                                                 |
| Direccion       | nombre_via, numero_edificacion, sector, codigo_postal, municipio, provincia |
| Estructura      | distribucion (torres   edificios), cantidad_edificios, cantidad_pisos       |
| Contacto        | telefono, correo                                                            |

| Grupo de campos | Contenido                                      |
|-----------------|------------------------------------------------|
| Otros           | mapa_iframe (TextField para incrustar un mapa) |

**NOTA****La distribucion es inmutable**

Residencial.save() consulta el valor original de distribucion y lo restaura si se intenta cambiar. La razon: una vez definida la estructura del residencial, cambiarla invalidaria los datos ya cargados. La UI muestra el campo como bloqueado. Consecuencia practica: si necesita cambiar la distribucion, actualice la fila directamente en la base de datos, no desde la aplicacion.

## 6.8 Inventario de modelos por aplicacion

Resumen de los 23 modelos del sistema, agrupados por aplicacion.

| App            | Modelos                                          | Cantidad |
|----------------|--------------------------------------------------|----------|
| accounts       | User (AbstractUser con PK UUID, login por email) | 1        |
| residencial    | Residencial (modelo de configuracion singular)   | 1        |
| structure      | Garden, Building, Apartment                      | 3        |
| propietarios   | Propietario                                      | 1        |
| residents      | Resident                                         | 1        |
| payments       | Payment                                          | 1        |
| maintenance    | MaintenanceCharge                                | 1        |
| incidents      | Incident, IncidentImage, IncidentHistory         | 3        |
| solicitudes    | Solicitud                                        | 1        |
| visitors       | Visitor                                          | 1        |
| reservations   | Reservation, CommonArea (obsoleto)               | 2        |
| areas_comunes  | AreaComun                                        | 1        |
| communications | Communication                                    | 1        |
| notifications  | Notification                                     | 1        |
| reports        | AuditLog, AuditLogDetail                         | 2        |
| chat           | ChatGroup, ChatMessage                           | 2        |
| login          | (sin modelos)                                    | 0        |
| inicio         | (sin modelos)                                    | 0        |

Tabla 6.5 - Modelos por aplicacion (23 en total).

## 6.9 Decisiones de diseno del modelo

### PROTECT en los vinculos con el usuario que registra

Varias relaciones usan `on_delete=PROTECT` para el usuario que creo el registro: incidencias (`reported_by`), visitantes (`registered_by`), comunicados (`sender`), pagos registrados y el historial de incidencias. La razon es de trazabilidad: no se debe poder eliminar un usuario que ha generado movimientos o cuya firma aparece en un documento.

### SET\_NULL donde la relacion es contextual

Los vinculos que enriquecen pero no definen el registro usan `SET_NULL`: el propietario de un apartamento, el cargo que aplica a todos, el garden de un usuario, la asignacion de una incidencia, la aprobacion de una reserva.

### CASCADE donde la dependencia es fuerte

Un apartamento que se elimina arrastra sus residentes, pagos, incidencias, solicitudes, visitantes y reservas. Un jardin arrastra sus edificios y, en cascada, sus apartamentos.

### Restricciones de integridad a nivel de base de datos

| Restriccion                                                       | Modelo                                                                                      | Efecto                                                     |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <code>unique_together (garden, name)</code>                       | Building                                                                                    | No se repiten nombres de edificio dentro de un jardin      |
| <code>unique_together (building, name)</code>                     | Apartment                                                                                   | No se repiten nombres de apartamento dentro de un edificio |
| <code>unique_together (garden, name)</code>                       | <code>reservations.CommonArea</code>                                                        | No se repiten areas dentro de un jardin                    |
| <code>UniqueConstraint (common_area, start_time, end_time)</code> | Reservation                                                                                 | Bloquea reservas exactamente duplicadas                    |
| <code>UniqueConstraint (incident, url)</code>                     | IncidentImage                                                                               | No se repite la misma URL en una incidencia                |
| <code>UniqueConstraint (audit_log, key)</code>                    | AuditLogDetail                                                                              | No se repite una clave de detalle en una bitacora          |
| <code>unique=True</code>                                          | User.email, User.document, Propietario.cedula, Resident.cedula, Garden.name, ChatGroup.name | Claves de negocio unicas                                   |

Tabla 6.6 - Restricciones declaradas a nivel de modelo.

#### ATENCION

##### La reserva no valida solapamientos parciales

La `UniqueConstraint` de `Reservation` solo impide duplicados exactos de (`common_area`, `start_time`, `end_time`). Dos reservas de 10:00 a 12:00 y de 11:00 a 13:00 en la misma area pasan la validacion de base de datos y colisionan. Los horarios de disponibilidad del AreaComun (`available_from` / `available_until` / `available_days`) tampoco se aplican a nivel de modelo: solo se validan en el codigo del serializer o de la vista.

## 6.10 Claves foraneas polimorficas manuales

Dos modelos usan un `UUIDField` suelto en lugar de una `ForeignKey` para apuntar a entidades de naturaleza variable. Esto evita una tabla puente, a costa de perder la integridad referencial:

| Modelo                                    | Campo                                       | Apunta a                                                                         | Riesgo                                 |
|-------------------------------------------|---------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------|
| <code>communications.Communication</code> | <code>target_id (UUID) + target_type</code> | <code>building_id</code> o <code>user_id</code> , segun <code>target_type</code> | Puede apuntar a un registro ya borrado |

| Modelo                     | Campo             | Apunta a                      | Riesgo                                 |
|----------------------------|-------------------|-------------------------------|----------------------------------------|
| notifications.Notification | related_id (UUID) | incident_id, payment_id, etc. | Puede apuntar a un registro ya borrado |

Tabla 6.7 - Referencias sin integridad referencial.

Si necesita integridad garantizada para estos vinculos, el camino es un patron de referencia generica (content\_type + object\_id) o un modelo puente por tipo.

## 7. API REST

### 7.1 Configuración global de DRF

```
# settings.py (bloque REST_FRAMEWORK que manda)
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_PAGINATION_CLASS':
        'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
}
```

| Ajuste              | Valor                               | Implicación                                           |
|---------------------|-------------------------------------|-------------------------------------------------------|
| Autenticación       | SessionAuthentication               | La API usa la cookie de sesión; no hay tokens         |
| Permiso por defecto | IsAuthenticated                     | Cualquier endpoint sin permiso explícito exige sesión |
| Paginación          | PageNumberPagination, 20 por página | Respuesta con count / next / previous / results       |
| Filtro              | DjangoFilterBackend                 | Campos declarados en filterset_fields                 |
| Busqueda            | SearchFilter                        | Campos declarados en search_fields                    |
| Orden               | OrderingFilter                      | Campos declarados en ordering_fields                  |

### 7.2 Patrón de los ViewSets

Los 21 ViewSets del sistema siguen el mismo esqueleto: queryset base filtrado por visibilidad, serializer (a veces elegido por acción), permiso de clase, y tres backends de filtro con sus campos declarados.

#### *Anatomía de un ViewSet típico*

```

class PaymentViewSet(viewsets.ModelViewSet):
    queryset = Payment.objects.all()
    serializer_class = PaymentSerializer
    permission_classes = [IsGestionOrSoloLectura]

    # Filtro por query string sobre campos del modelo
    filterset_fields = ['apartment', 'resident', 'status',
                       'payment_method', 'concept']
    # Búsqueda de texto libre
    search_fields = ['resident__full_name', 'apartment__number',
                    'resident__cedula']
    # Campos por los que se puede ordenar
    ordering_fields = ['period', 'payment_date', 'status']

    def get_queryset(self):
        # El listado se restringe a lo que le corresponde al usuario
        qs = _pagos_visibles(self.request.user)
        return qs.distinct()

```

## 7.3 Inventario de endpoints

Cada aplicación monta su propio `DefaultRouter`. No existe un prefijo global `/api/`: los endpoints cuelgan del prefijo de la app.

| App            | Prefijo                       | ViewSet                               | Acciones propias                        |
|----------------|-------------------------------|---------------------------------------|-----------------------------------------|
| accounts       | /cuentas/                     | UserViewSet                           | login, profile, change_password, logout |
| residencial    | /residencial/residencial/     | ResidencialViewSet                    | -                                       |
| structure      | /departamentos/jardines/      | GardenViewSet                         | -                                       |
| structure      | /departamentos/edificios/     | BuildingViewSet                       | -                                       |
| structure      | /departamentos/departamentos/ | ApartmentViewSet                      | -                                       |
| propietarios   | /propietarios/                | PropietarioViewSet                    | -                                       |
| residents      | /residentes/                  | ResidentViewSet                       | -                                       |
| payments       | /pagos/registros/             | PaymentViewSet                        | -                                       |
| maintenance    | /mantenimientos/              | MaintenanceChargeViewSet              | -                                       |
| incidents      | /incidencias/incidencias/     | IncidentViewSet                       | -                                       |
| incidents      | /incidencias/historial/       | IncidentHistoryViewSet (solo lectura) | -                                       |
| solicitudes    | /solicitudes/                 | SolicitudViewSet                      | -                                       |
| visitors       | /visitantes/registros/        | VisitorViewSet                        | authorize, reject, check_in, check_out  |
| reservations   | /reservas/areas-comunes/      | CommonAreaViewSet                     | -                                       |
| reservations   | /reservas/                    | ReservationViewSet                    | -                                       |
| areas_comunes  | /areas-comunes/api/           | AreaComunViewSet                      | -                                       |
| communications | /comunicados/                 | CommunicationViewSet                  | -                                       |
| notifications  | /notificaciones/              | NotificationViewSet                   | -                                       |

| App     | Prefijo                              | ViewSet                        | Acciones propias             |
|---------|--------------------------------------|--------------------------------|------------------------------|
| chat    | /chat/                               | ChatMessageViewSet             | -                            |
| reports | /reportes/bitacoras/                 | AuditLogViewSet (solo lectura) | -                            |
| reports | /reportes/{resumen,pagos,ocupacion}/ | ReportViewSet (acciones)       | summary, payments, occupancy |

Tabla 7.1 - Endpoints registrados por router.

## 7.4 Operaciones CRUD generadas por el router

Cada ModelViewSet expone automaticamente las operaciones estandar. El nombre de la URL aparece en el campo name de cada ruta del router:

| Operacion      | Metodo HTTP | Ruta           | Nombre en el router         |
|----------------|-------------|----------------|-----------------------------|
| Listar         | GET         | /prefijo/      | <basename>-list             |
| Crear          | POST        | /prefijo/      | <basename>-list             |
| Detalle        | GET         | /prefijo/<pk>/ | <basename>-detail           |
| Actualizar     | PUT / PATCH | /prefijo/<pk>/ | <basename>-detail           |
| Borrar         | DELETE      | /prefijo/<pk>/ | <basename>-detail           |
| Raiz de la API | GET         | /prefijo/      | <basename>-root (navegador) |

El <pk> es el UUID del objeto. Las operaciones de creacion, actualizacion y borrado solo funcionan si el permiso asignado a la accion lo permite.

## 7.5 Acciones personalizadas

### Autenticacion y perfil (accounts)

| Accion          | Metodo | Ruta                      | Permiso         | Que hace                                   |
|-----------------|--------|---------------------------|-----------------|--------------------------------------------|
| login           | POST   | /cuentas/login/           | AllowAny        | Autentica por email y contrasena           |
| profile         | GET    | /cuentas/profile/         | IsAuthenticated | Devuelve el perfil del usuario autenticado |
| change_password | POST   | /cuentas/change_password/ | IsAuthenticated | Cambia la contrasena validando la actual   |
| logout          | POST   | /cuentas/logout/          | IsAuthenticated | Cierra la sesion                           |

### Ciclo de vida de un visitante (visitors)

El control de acceso de porteria se implementa como acciones de detalle. Todas exigen POST y devuelven 403 si el usuario no es de seguridad o gestion.

| Accion    | Ruta                                       | Estado requerido     | Resultado                                  |
|-----------|--------------------------------------------|----------------------|--------------------------------------------|
| authorize | POST /visitantes/registros/<pk>/authorize/ | pending              | Pasa a authorized y registra authorized_by |
| reject    | POST /visitantes/registros/<pk>/reject/    | pending              | Pasa a rejected                            |
| check_in  | POST /visitantes/registros/<pk>/check_in/  | authorized           | Registra actual_entry y pasa a completed   |
| check_out | POST /visitantes/registros/<pk>/check_out/ | actual_entry no nula | Registra actual_exit y pasa a completed    |

Tabla 7.2 - Transiciones de estado de un visitante.

**NOTA****Estados del visitante**

Flujo normal: pending (esperando) -> authorized (autorizado) -> check\_in registra la entrada real y check\_out la salida. En cualquier momento antes de la entrada se puede rechazar.

**Reportes (reports)**

| Accion    | Ruta                     | Que devuelve                                                                                                                      |
|-----------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| summary   | GET /reportes/resumen/   | 14 metricas: totales de apartamentos, ocupados, residentes activos, pagos, incidencias, visitantes, reservas e ingresos mensuales |
| payments  | GET /reportes/pagos/     | Conteo e importes por estado de pago; acepta <apartment>, <status>, <month>, <year>                                               |
| occupancy | GET /reportes/ocupacion/ | Conteo de apartamentos por estado (vacio, ocupado, etc.)                                                                          |

Tabla 7.3 - Reportes accesibles por API.

**7.6 Filtros, busqueda y ordenamiento**

Los tres backends conviven en la misma respuesta. Se combinan en un unico query string:

**Combinando filtros en una consulta**

```
GET /pagos/registros/
?apartment=<uuid> DjangoFilterBackend (filtro exacto)
&status=overdue DjangoFilterBackend (filtro exacto)
&search=juan SearchFilter (busqueda en search_fields)
&ordering=-period OrderingFilter (campo en ordering_fields)
&page=2 PageNumberPagination (20 por pagina)
```

Los search\_fields se pueden recorrer en relaciones (resident\_\_full\_name significa buscar en el nombre del residente del pago). Los ordering\_fields controlan que campos se pueden ordenar via query string; un campo no listado alli no es ordenable.

**7.7 Visibilidad en la API**

Los ViewSets no se limitan al permiso de clase: ademas recortan el queryset para que un usuario no pueda ver mas de lo que le corresponde, aunque tenga permiso de lectura. El recorte se hace en get\_queryset() con un filtro por usuario.

**Recorte de queryset por usuario**

```
def get_queryset(self):
    user = self.request.user
    if user.role in ('admin', 'manager'):
        return Model.objects.all() # gestion ve todo
    return Model.objects.filter(<lo que le toca>).distinct()
```

**7.8 Formato de las respuestas**

Una peticion de listado devuelve la estructura de PageNumberPagination:

### ***Estructura de una respuesta paginada***

```
{
  "count": 137, // total de registros que cumplen el filtro
  "next": "http://...page=2", // URL de la siguiente pagina (null si es la ultima)
  "previous": null, // URL de la anterior (null si es la primera)
  "results": [ // la pagina actual de resultados
    { "id": "...", "...": "..." },
    ...
  ]
}
```

Los errores de validacion devuelven el detalle por campo; los errores de permiso devuelven 403; los de autenticacion, 401 (o 403 con sesion).

## 8. Frontend: plantillas, CSS y JavaScript

### 8.1 Arquitectura del frontend

El frontend es un servidor renderizado clásico, sin framework de JavaScript. Cada pagina se renderiza en el servidor con plantillas de Django, y el JavaScript externa solo anade interactividad (dialogos, filtros asincronos, confirmaciones, mascaras de entrada). No hay paso de build, ni bundler, ni npm.

### 8.2 Plantillas

#### Plantilla base y componentes

Toda la interfaz se apoya en `templates/base.html`, que incluye los componentes reutilizables del directorio `templates/componentes/`:

| Componente                         | Funcion                                                                            |
|------------------------------------|------------------------------------------------------------------------------------|
| <code>base.html</code>             | Esqueleto comun: head, estilos, barra superior, menu lateral y bloque de contenido |
| <code>header.html</code>           | Cabecera superior con el logotipo y el menu de usuario                             |
| <code>aside.html</code>            | Menu lateral de navegacion entre modulos (respeta los permisos del usuario)        |
| <code>menu_usuario.html</code>     | Desplegable del perfil con opcion de cerrar sesion y editar perfil                 |
| <code>modal.html</code>            | Estructura base de los dialogos (modales)                                          |
| <code>botones.html</code>          | Botones de accion reutilizables (guardar, cancelar, eliminar)                      |
| <code>mini_card.html</code>        | Tarjeta compacta para el dashboard                                                 |
| <code>paginacion_tabla.html</code> | Controles de paginacion reutilizables                                              |
| <code>login.html</code>            | Estructura de la pantalla de inicio de sesion                                      |

Tabla 8.1 - Componentes globales de plantilla.

#### Plantillas por modulo

Cada modulo de dominio aporta sus propias plantillas en `<app>/templates/<app>/`. La pestana principal de cada modulo se llama siempre `index.html`; los formularios suelen estar en `agregar.html` o `nuevo.html`.

| Vista             | Plantilla                                           | Contexto tipico                                                                  |
|-------------------|-----------------------------------------------------|----------------------------------------------------------------------------------|
| Listado           | <code>index.html</code>                             | <code>page_obj</code> (paginacion), objeto, filtros, puede_crear/editar/eliminar |
| Formulario nuevo  | <code>agregar.html</code> / <code>nuevo.html</code> | form (ModelForm), titulo, accion                                                 |
| Formulario editar | <code>agregar.html</code> / <code>nuevo.html</code> | form, instancia, <code>editing=True</code>                                       |
| Perfil            | <code>perfil.html</code>                            | form de datos personales                                                         |

### 8.3 Sistema de diseno: `variables.css`

Los estilos se organizan en torno a un archivo de variables que es la fuente unica de verdad del diseno. `static/css/variables.css` define todos los tokens (colores, sombras, bordes, radios, espaciados) y se importa

desde las hojas base.

## Paleta de colores

El sistema usa una paleta de verdes como color de marca, mas una escala de grises para superficies y texto, y una escala de azules para elementos informativos. Los nombres siguen el patron `--<familia>-<tono>`.

| Familia               | Ejemplo de variables                                                           | Uso                                                    |
|-----------------------|--------------------------------------------------------------------------------|--------------------------------------------------------|
| Verdes (marca)        | <code>--green-500</code> , <code>--green-700</code> , <code>--green-900</code> | Color primario, enlaces, acciones, estados activos     |
| Blancos (superficies) | <code>--white-50</code> a <code>--white-900</code>                             | Fondos de pagina, tarjetas, superficies elevadas       |
| Negros (texto)        | <code>--black-300</code> , <code>--black-500</code> , <code>--black-900</code> | Texto principal, texto secundario, texto deshabilitado |
| Azules (informativo)  | <code>--blue-500</code> , <code>--blue-600</code>                              | Mensajes informativos, enlaces, foco                   |

Tabla 8.2 - Familias de color del sistema de diseno.

## Reglas de uso de las variables

- Todo color, sombra, borde, radio, margen y padding se escribe con una variable, nunca como valor literal repetido.
- La unica unidad px permitida es para bordes y sombras (los tokens `--border-*`, `--borderR-*` y `--shadow-*` ya estan en px).
- Cualquier otra medida (margen, padding, tamano, tipografia, ancho, alto) se escribe en rem. Con `font-size: 62.5% en :root`, 1rem equivale a 10px.
- Los breakpoints de las media queries se mantienen en px (560px y 900px), que es la practica estandar.
- Quien use variables debe asegurar que su hoja las importe antes (directamente o via `style.css`).

### BUENA PRACTICA

#### Por que importa

Centralizar el diseno en variables permite cambiar la identidad visual completa del sistema editando un solo archivo, y garantiza coherencia entre modulos. Es la diferencia entre un sistema mantenible y veinte paginas que se parecen pero no encajan.

## 8.4 Organizacion de los estilos

| Archivo                                 | Ambito | Contenido                                                  |
|-----------------------------------------|--------|------------------------------------------------------------|
| <code>static/css/variables.css</code>   | Global | Tokens de diseno (fuente unica)                            |
| <code>static/css/style.css</code>       | Global | Estilos base; importa <code>variables.css</code>           |
| <code>static/css/formularios.css</code> | Global | Secciones, filas, campos, interruptores de los formularios |
| <code>static/css/tablas.css</code>      | Global | Estilo de las tablas de datos                              |
| <code>static/css/paginacion.css</code>  | Global | Controles de paginacion                                    |
| <code>static/css/dialogos.css</code>    | Global | Modales y dialogos                                         |
| <code>static/css/acciones.css</code>    | Global | Paneles de acciones del dashboard                          |
| <code>static/css/entradas.css</code>    | Global | Campos de entrada                                          |

| Archivo                   | Ambito | Contenido                      |
|---------------------------|--------|--------------------------------|
| static/css/mediaquers.css | Global | Breakpoints comunes            |
| <app>/static/css/*.css    | Modulo | Estilos especificos del modulo |

Tabla 8.3 - Organizacion de las hojas de estilo.

La separacion es deliberada: lo que comparten dos o mas modulos vive en `static/css/` del proyecto; lo que solo usa un modulo, en su propia carpeta `static/`.

## 8.5 Convenciones de nombres en CSS

Toda la nomenclatura de clases esta en espanol, con nombres claros y concisos. El proyecto mantiene un mapeo canonico desde los nombres genericos originales a los nombres en espanol:

| Nombre original         | Nombre canonico en el proyecto       |
|-------------------------|--------------------------------------|
| form-section            | seccion-formulario                   |
| form-row                | fila-formulario                      |
| form-field              | campo-formulario                     |
| select-field            | campo-seleccion                      |
| input-field             | campo-entrada                        |
| form-switch-row         | fila-interruptor                     |
| switch-label            | etiqueta-interruptor                 |
| switch / switch-slider  | interruptor / deslizador-interruptor |
| switch-input            | entrada-interruptor                  |
| dialog-header           | cabecera-dialogo                     |
| dialog-footer           | pie-formulario                       |
| dialog-close            | boton-cerrar                         |
| btn-generar-comprobante | btn-comprobante                      |
| Acciones-dashboard      | panel-acciones                       |
| container-acciones      | contenedor-acciones                  |
| pag-btn                 | boton-pagina                         |
| pag-num                 | numero-pagina                        |
| paginacion-numeros      | numeros-paginacion                   |

Tabla 8.4 - Mapeo de nombres de clase.

Los nombres de archivo siguen la misma linea: los estilos especificos se llaman como el modulo y en espanol (pagos.css, departamentos.css, incidencias.css, visitantes.css).

## 8.6 Reglas obligatorias del frontend

**PENDIENTE / BUG****Reglas de refactor frontend (obligatorias)**

1. Nada de `<script>` inline ni estilos `style=` en el HTML. El unico `<script>` permitido es `<script src='...'>` para conectar el HTML con su JS. Los manejadores (onchange, onclick, ...) van dentro del archivo JS externo.
2. Estilos y JS compartidos entre apps viven en `static/css/` y `static/js/` del proyecto. Lo especifico de un modulo queda en su `static/` propio.
3. El HTML principal de cada modulo se llama siempre `index.html`; los demas archivos se nombran por su contenido.
4. CSS en español, con nombres claros y concisos. Los valores salen de `variables.css`; `px` solo para bordes y sombras; `rem` para todo lo demas.
5. El JS externo no puede usar etiquetas de plantilla Django. El HTML le pasa los valores con atributos `data-*` (`data-url-comprobante`, `data-pago-apartamento`, `data-departamento`).
6. Mantener la responsividad: los estilos compartidos incluyen media queries (breakpoints 560px / 900px) usados por formularios y por la rejilla de acciones.

## 8.7 JavaScript

### Organizacion

| Archivo                                       | Ambito | Responsabilidad                                  |
|-----------------------------------------------|--------|--------------------------------------------------|
| <code>static/js/dialogos.js</code>            | Global | Apertura, cierre y control de los modales        |
| <code>static/js/pagos.js</code>               | Global | Logica compartida del modulo de pagos            |
| <code>static/js/mascaras.js</code>            | Global | Mascaras de entrada (telefono, cedula, montants) |
| <code>static/js/contadores.js</code>          | Global | Contadores animados del dashboard                |
| <code>static/js/menu-movil.js</code>          | Global | Comportamiento del menu en movil                 |
| <code>static/js/usuario-menu.js</code>        | Global | Desplegable del menu de usuario                  |
| <code>static/js/encabezado-acciones.js</code> | Global | Barra de acciones superior                       |
| <code>&lt;app&gt;/static/js/*.js</code>       | Modulo | Comportamiento especifico del modulo             |

### Como el JS recibe datos del HTML

El punto de friccion clasico entre plantillas Django y JavaScript son las URLs y los identificadores. La solucion del proyecto es pasar los valores del servidor al script mediante atributos `data-*`, de modo que el JS nunca tenga que interpretar una etiqueta de plantilla:

**Puente HTML -> JavaScript con `data-*`**

```
<!-- HTML: el servidor pasa los valores al navegador por data-* -->
<button data-url-comprobante="{% url 'generar_comprobante' %}"
        data-pago-apartamento="{{ pago.apartment.id }}"
        data-departamento="{{ pago.apartment.name }}">
    Generar comprobante
</button>

// JS externo: lee de los data-* (sin etiquetas de plantilla)
document.querySelectorAll('[data-url-comprobante]').forEach(function (btn) {
    btn.addEventListener('click', function () {
        var url = btn.dataset.urlComprobante + '?pago_id=' + btn.dataset.pagoApartamento;
        window.open(url, '_blank');
    });
});
```

### ATENCION

#### Por que esta regla existe

Si el JS contuviera {% url %} o {{ var }}, Django lo renderizaria como texto literal dentro del archivo .js y el script fallaria en tiempo de ejecucion. Los data-\* evitan por completo ese problema y mantienen el JS reutilizable entre paginas.

## 8.8 Accesibilidad y experiencia de usuario

El proyecto aplica varias practicas de usabilidad de forma consistente:

- Confirmacion antes de acciones destructivas (eliminar) mediante dialogos.
- Mensajes de exito y de error mediante el sistema de mensajes de Django, renderizados como notificaciones.
- Tablas ordenables y paginadas para manejar volumen de datos.
- Formularios con validacion en servidor (ModelForm) y mascaras de entrada en el cliente.
- Estados vacios y de carga considerados en los listados.
- Menu lateral que se adapta a movil mediante JavaScript y media queries.
- Diseno responsivo con dos breakpoints principales (560px y 900px).

## 9. Servidor ASGI, tiempo real y tareas asincronas

### 9.1 Modelo de despliegue previsto

CONDYSOS esta disenado para correr sobre una pila ASGI con Daphne como servidor principal. La entrada es `condosys/asgi.py`.

### 9.2 Estado actual de ASGI

El punto de entrada ASGI es deliberadamente minimo: solo expone la aplicacion Django como callable, sin enrutar websockets. Ese es el unico motivo por el que los websockets no funcionan, aunque el codigo de channels este escrito.

#### *condosys/asgi.py*

```
# condosys/asgi.py (completo)
import os
from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'condosys.settings')

application = get_asgi_application()
# ^ Solo Django. No hay ProtocolTypeRouter.
```

Existe un modulo de routing completo en `chat/routing.py`, y los consumers estan implementados en `chat/consumers.py`. Las rutas definidas son:

```
# chat/routing.py
websocket_urlpatterns = [
    re_path(r'ws/chat/(?P<user_id>[\w-]+)/$',
        consumers.ChatConsumer.as_asgi()),
    re_path(r'ws/group/(?P<group_name>[\w-]+)/$',
        consumers.GroupChatConsumer.as_asgi()),
    re_path(r'ws/notifications/(?P<user_id>[\w-]+)/$',
        consumers.NotificationConsumer.as_asgi()),
]
```

### 9.3 Que falta para activar el tiempo real

| Paso | Que hacer                                                                                            | Archivo                    |
|------|------------------------------------------------------------------------------------------------------|----------------------------|
| 1    | Añadir 'channels' a INSTALLED_APPS                                                                   | settings.py                |
| 2    | Envolver la aplicacion en un ProtocolTypeRouter con 'http' y 'websocket', usando AuthMiddlewareStack | condosys/asgi.py           |
| 3    | Incluir las rutas de websocket (authMiddlewareStack(URLRouter(...)))                                 | condosys/asgi.py           |
| 4    | Verificar que el channel layer (Redis) este accesible                                                | settings.py CHANNEL_LAYERS |
| 5    | Probar el handshake del websocket desde el cliente JS                                                | cliente                    |

Tabla 9.1 - Pasos para activar los WebSockets.

#### **Estructura objetivo (referencia)**

```
# Estructura objetivo de asgi.py
from channels.auth import AuthMiddlewareStack
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.security.websocket import AllowedHostsOriginValidator
from django.core.asgi import get_asgi_application
from chat import routing as chat_routing

django_asgi_app = get_asgi_application()

application = ProtocolTypeRouter({
    'http': django_asgi_app,
    'websocket': AllowedHostsOriginValidator(
        AuthMiddlewareStack(
            URLRouter(chat_routing.websocket_urlpatterns)
        )
    ),
})
```

#### PENDIENTE / BUG

##### Punto ciego de seguridad

Los consumers existentes usan `self.scope['user']`, que solo existe si la conexion pasa por `AuthMiddlewareStack`. Sin ese middleware, el usuario no se puede resolver y el consumer fallaria al autenticar.

El `AllowedHostsOriginValidator` es igualmente necesario en produccion para evitar conexiones websocket desde origenes no autorizados.

## 9.4 Tareas asincronas (Celery)

Celery esta configurado en settings (broker y backend Redis, serializacion JSON, zona horaria America/Bogota) pero no hay aplicacion Celery implementada: no existe el archivo `celery.py` ni el inicializador de la app. Los casos de uso naturales si se quisiera activar son: envio de notificaciones por correo, avisos de pago por vencer y limpieza de sesiones.

## 10. Despliegue en produccion

### 10.1 Requisitos de produccion

| Componente            | Papel en produccion                                   |
|-----------------------|-------------------------------------------------------|
| Gunicorn              | Servidor WSGI de aplicacion                           |
| Daphne (ASGI)         | Servidor para el tiempo real (cuando se cablee)       |
| WhiteNoise            | Servido de archivos estaticos desde el propio proceso |
| PostgreSQL            | Base de datos de produccion                           |
| Nginx / reverse proxy | Terminacion TLS y proxy hacia Gunicorn                |
| Redis                 | Channel layer y broker de Celery                      |

### 10.2 Checklist de despliegue

| #  | Paso                       | Comando / accion                                                                  |
|----|----------------------------|-----------------------------------------------------------------------------------|
| 1  | Rama estable               | git checkout main && git pull                                                     |
| 2  | Entorno virtual            | python -m venv venv && venv\Scripts\activate                                      |
| 3  | Dependencias               | python -m pip install -r requirements.txt                                         |
| 4  | Archivo .env de produccion | Configurar DEBUG=False, SECRET_KEY fuerte, ALLOWED_HOSTS del dominio, CORS y CSRF |
| 5  | Migraciones                | python manage.py migrate                                                          |
| 6  | Base de datos              | Cambiar DATABASES a PostgreSQL y descomentar el bloque                            |
| 7  | Estaticos                  | python manage.py collectstatic --noinput                                          |
| 8  | Verificacion               | python manage.py check --deploy                                                   |
| 9  | Superusuario               | python manage.py createsuperuser                                                  |
| 10 | Arranque                   | gunicorn condosys.wsgi:application (con Daphne si hay websockets)                 |

Tabla 10.1 - Secuencia de despliegue.

#### ATENCION

##### No olvide collectstatic

En DEBUG, Django sirve los estaticos desde las carpetas de cada app. En produccion, con DEBUG=False, Django no los sirve: si no ejecuta collectstatic, la aplicacion arranca pero aparece sin estilos ni JavaScript.

### 10.3 Conmutar a PostgreSQL

El bloque de PostgreSQL esta listo y comentado en settings.py. Para activarlo sustituya el bloque de SQLite por el bloque de PostgreSQL y configure las variables en .env.

```
# settings.py - activar el bloque de PostgreSQL
DATABASES = {
    'default': {
        'ENGINE': os.getenv('DATABASE_ENGINE',
        'django.db.backends.postgresql'),
        'NAME': os.getenv('DATABASE_NAME', 'condosys_db'),
        'USER': os.getenv('DATABASE_USER', 'postgres'),
        'PASSWORD': os.getenv('DATABASE_PASSWORD', ''),
        'HOST': os.getenv('DATABASE_HOST', 'localhost'),
        'PORT': os.getenv('DATABASE_PORT', '5432'),
        'ATOMIC_REQUESTS': True,
    }
}
```

- Instale el driver con: `python -m pip install psycopg2` (ya esta en requirements).
- Cree la base de datos y el usuario en PostgreSQL antes de migrar.
- `ATOMIC_REQUESTS=True` envuelve cada peticion en una transaccion, lo que garantiza consistencia pero puede afectar al rendimiento con operaciones pesadas.

## 10.4 Seguridad en produccion

settings.py define un bloque de endurecimiento que se activa cuando `DEBUG=False`. Cada variable se lee del `.env`, de modo que puede ajustarse por entorno:

| Ajuste                                      | Proposito                                                      |
|---------------------------------------------|----------------------------------------------------------------|
| <code>SECURE_SSL_REDIRECT</code>            | Redirigir todo el trafico HTTP a HTTPS                         |
| <code>SECURE_HSTS_SECONDS</code>            | Duracion del HSTS (31536000 = 1 ano)                           |
| <code>SECURE_HSTS_INCLUDE_SUBDOMAINS</code> | Incluir subdominios en HSTS                                    |
| <code>SECURE_HSTS_PRELOAD</code>            | Permitir pre-carga del HSTS en el navegador                    |
| <code>SESSION_COOKIE_SECURE</code>          | Cookie de sesion solo por HTTPS                                |
| <code>CSRF_COOKIE_SECURE</code>             | Cookie CSRF solo por HTTPS                                     |
| <code>SECURE_CONTENT_TYPE_NOSNIFF</code>    | Prevenir el olfateo de tipos de contenido                      |
| <code>SECURE_REFERRER_POLICY</code>         | Politica de referencia (same-origin)                           |
| <code>CSRF_TRUSTED_ORIGINS</code>           | Origenes de confianza para formularios (dominio de produccion) |
| <code>CORS_ALLOWED_ORIGINS</code>           | Origenes permitidos si hay consumos externos                   |

Tabla 10.2 - Ajustes de seguridad disponibles por entorno.

### PENDIENTE / BUG

#### SECRET\_KEY no debe quedar en el repositorio

La clave secreta se lee del `.env` y `.env` esta en `.gitignore`, lo cual es correcto. Verifique que ninguna copia de la clave se haya committeado por error antes de publicar el repositorio.

## 10.5 Lista de verificacion de seguridad antes de publicar

|    | Verificacion                                         | Estado |
|----|------------------------------------------------------|--------|
| 1  | DEBUG=False en el .env de produccion                 | [ ]    |
| 2  | SECRET_KEY unica, larga y no versionada              | [ ]    |
| 3  | ALLOWED_HOSTS limitado a los dominios reales         | [ ]    |
| 4  | ALLOWED_HOSTS no incluye comodines en produccion     | [ ]    |
| 5  | CSRF_TRUSTED_ORIGINS configurado con el dominio real | [ ]    |
| 6  | SECURE_SSL_REDIRECT activo (sitio bajo HTTPS)        | [ ]    |
| 7  | Cookies de sesion y CSRF marcadas como seguras       | [ ]    |
| 8  | collectstatic ejecutado y servido por WhiteNoise     | [ ]    |
| 9  | manage.py check --deploy sin avisos criticos         | [ ]    |
| 10 | Datos de ejemplo (poblar_datos) eliminados           | [ ]    |
| 11 | Contraseñas del superusuario cambiadas               | [ ]    |
| 12 | Redis con autenticacion y no expuesto a Internet     | [ ]    |

## 11. Problemas conocidos y deuda tecnica

Este capitulo documenta los defectos y limitaciones que hoy existen en el codigo. Se incluye de forma deliberada: conocerlos antes de tocar el sistema evita descubrimientos en produccion. Cada punto indica donde esta el problema y cual es el camino de correccion.

### 11.1 Configuracion (settings.py)

| # | Problema                                                                                                                              | Ubicacion   | Correccion sugerida                                   |
|---|---------------------------------------------------------------------------------------------------------------------------------------|-------------|-------------------------------------------------------|
| 1 | Bloque REST_FRAMEWORK duplicado: el primero define throttling y el segundo lo sobrescribe. El throttling configurado nunca se aplica. | settings.py | Consolidar en un solo bloque, con throttling incluido |
| 2 | AUTH_USER_MODEL declarado dos veces                                                                                                   | settings.py | Dejar una sola declaracion                            |
| 3 | STATIC_URL, STATICFILES_DIRS y STATIC_ROOT definidos dos veces                                                                        | settings.py | Eliminar el bloque duplicado                          |
| 4 | El bloque de seguridad if not DEBUG es sobrescrito por el bloque posterior que lee del .env                                           | settings.py | Unificar en una sola lectura por entorno              |
| 5 | SECRET_KEY sin valor por defecto; el import de ImproperlyConfigured no se usa                                                         | settings.py | Añadir fallback seguro o error explicito              |

Tabla 11.1 - Duplicaciones y omisiones en la configuracion.

### 11.2 URLs

| # | Problema                                                                                                   | Efecto                                                                                                                                                                        | Correccion sugerida                                                                |
|---|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 5 | En 9 apps, la vista HTML app_index esta registrada en " antes de include(router.urls) con el router en r". | El listado HTML tapa el listado JSON: los endpoints -list y -root de la API quedan inalcanzables en la raiz del prefijo. El -detail y las acciones detail=False si funcionan. | Mover el router a un prefijo propio (por ejemplo api/), como ya hace areas_comunes |

Tabla 11.2 - Sombreo de rutas entre HTML y API.

Apps afectadas: accounts, propietarios, residents, maintenance, solicitudes, areas\_comunes (parcialmente), reservations, communications, notifications y chat.

### 11.3 Tiempo real

| # | Problema                           | Efecto                                                        | Correccion sugerida                          |
|---|------------------------------------|---------------------------------------------------------------|----------------------------------------------|
| 6 | asgi.py no usa ProtocolTypeRouter  | Los WebSockets de chat y notificaciones no se pueden conectar | Cablear el routing (ver capitulo 9)          |
| 7 | channels no esta en INSTALLED_APPS | Channels no inicializa su app                                 | Añadir la app                                |
| 8 | Consumers sin AuthMiddlewareStack  | self.scope['user'] no se resuelve                             | Envolver el URLRouter en AuthMiddlewareStack |

Tabla 11.3 - Estado del tiempo real.

### 11.4 Modelo de datos

| #  | Problema                                                                                                                                                                                                                                         | Ubicacion                     | Correccion sugerida                                |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|----------------------------------------------------|
| 9  | chat.ChatMessage.__str__ referencia self.group_name, campo que no existe: lanza AttributeError en todo mensaje grupal                                                                                                                            | chat/models.py                | Usar self.group.name y proteger el caso group nulo |
| 10 | Dos modelos para el area comun: reservations.CommonArea (con garden) y areas_comunes.AreaComun (sin garden). CommonArea quedo huérfano tras la migración 0003, pero sigue registrado en admin, forms, serializers, ViewSet, router y en el seed. | reservations/, areas_comunes/ | Eliminar CommonArea y migrar sus referencias       |
| 11 | AreaComun no tiene relacion con Garden ni con Residencial: no se puede asignar un area a un jardin o edificio concreto                                                                                                                           | areas_comunes/models.py       | Añadir la FK a Garden                              |
| 12 | PK no uniforme: chat.ChatGroup, incidents.IncidentImage y reports.AuditLogDetail usan AutoField                                                                                                                                                  | varios                        | Migrar a UUIDField                                 |
| 13 | Import muerto: Building en communications/models.py; PROTECT sin usar en accounts y structure                                                                                                                                                    | varios                        | Limpiar                                            |
| 14 | ChatGroup no define clase Meta                                                                                                                                                                                                                   | chat/models.py                | Añadir ordering y verbose_name_plural              |
| 15 | FK polimórficas manuales sin integridad (Communication.target_id, Notification.related_id)                                                                                                                                                       | varios                        | Referencia genérica o tabla puente                 |

Tabla 11.4 - Inconsistencias del modelo de datos.

## 11.5 Seguridad y permisos

| #  | Problema                                                                                                                                 | Efecto                                                                                  |
|----|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| 16 | El middleware de CORS esta activo pero la app corsheaders no esta en INSTALLED_APPS                                                      | Las cabeceras CORS probablemente no se aplican                                          |
| 17 | El throttling de DRF esta configurado en el bloque que se sobreescribe                                                                   | No hay limite de peticiones a la API                                                    |
| 18 | Permisos declarados pero sin usar: IsResidentOnly, IsPropietario, IsResidentOrManager, IsSecurity, CanModifyReservation                  | Código muerto; IsResidentOrManager duplica a IsResident                                 |
| 19 | Permisos de objeto aplicados como permiso de clase sin has_permission: CanAccessApartment y CanModifyIncident no restringen los listados | El recorte real depende del filtro de get_queryset, no del permiso                      |
| 20 | Los mensajes de chat no validan rol (solo login_required)                                                                                | Cualquier usuario autenticado puede usar el chat (consistente con la tabla de permisos) |

Tabla 11.5 - Hallazgos de seguridad y permisos.

## 11.6 Calidad y pruebas

| #  | Problema                                                                | Efecto                                        | Riesgo |
|----|-------------------------------------------------------------------------|-----------------------------------------------|--------|
| 21 | Los tests.py de todas las apps son plantillas vacías; no hay suite real | Ninguna regresión se detecta automáticamente  | Alto   |
| 22 | No hay linter, formateador ni integración continua                      | El estilo depende de la disciplina individual | Medio  |
| 23 | requirements.txt sin versiones fijadas                                  | Instalaciones no reproducibles                | Medio  |

| #  | Problema                                                                                                                        | Efecto                                                           | Riesgo |
|----|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|--------|
| 24 | Constantes duplicadas entre apps: MARITAL_CHOICES en propietarios y residents; PAYMENT_METHOD_CHOICES en payments y maintenance | Cambiar una exige cambiar la otra                                | Bajo   |
| 25 | El atributo filter_fields (obsoleto en DRF moderno) se usa en accounts.UserViewSet                                              | Se ignora silenciosamente; el filtro por rol/estado no se aplica | Medio  |
| 26 | Envio de datos de demostracion: el comando poblar_datos crea usuarios con contrasena conocida                                   | Riesgo si se ejecuta en produccion                               | Alto   |

Tabla 11.6 - Deuda tecnica y riesgos de calidad.

## 11.7 Orden de trabajo sugerido

Si va a abordar la deuda tecnica, este orden minimiza el riesgo:

| Prioridad | Tarea                                                                       | Por que primero                                   |
|-----------|-----------------------------------------------------------------------------|---------------------------------------------------|
| 1         | Eliminar los datos de ejemplo del flujo de despliegue y proteger el comando | Riesgo de seguridad directo                       |
| 2         | Arreglar el sombreado de URLs moviendo los routers a /api/                  | Devuelve la API a la normalidad sin romper la UI  |
| 3         | Consolidar los bloques duplicados de settings.py                            | Reduce la confusion sobre que configuracion manda |
| 4         | Activar o retirar channels/Celery                                           | Decidir si son capacidad real o codigo muerto     |
| 5         | Arreglar ChatMessage.__str__ y limpiar permisos sin uso                     | Correccion de bajo riesgo, alto valor             |
| 6         | Escribir la suite de pruebas minima (permisos y logica de pagos)            | Sin esto, cualquier refactor grande es arriesgado |
| 7         | Congelar versiones en requirements.txt                                      | Reproducibilidad del despliegue                   |
| 8         | Unificar los modelos de area comun                                          | Limpieza estructural, hacerla antes de crecer     |

Tabla 11.7 - Backlog tecnico priorizado.

## 12. Guia de mantenimiento y extension

### 12.1 Como agregar un modulo nuevo

El sistema esta disenado para que agregar un modulo sea mecanico. Los pasos:

| #  | Paso                            | Detalle                                                                                                                                   |
|----|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Crear la app                    | python manage.py startapp (en espanol, en plural)                                                                                         |
| 2  | Registrar la app                | Anadir " a INSTALLED_APPS en settings.py                                                                                                  |
| 3  | Definir el modelo               | models.py con UUIDField como PK, related_name en todas las FK, indices en los campos de filtro                                            |
| 4  | Crear migraciones               | python manage.py makemigrations && python manage.py migrate                                                                               |
| 5  | Definir serializers             | serializers.py con el serializer de listado y, si difiere, el de detalle                                                                  |
| 6  | Definir el ViewSet              | views.py con el ViewSet, permission_classes, filterset_fields, search_fields, ordering_fields y get_queryset con el filtro de visibilidad |
| 7  | Definir permisos                | Usar los permisos de accounts.permissions; si hace falta uno nuevo, crearlo alli                                                          |
| 8  | Definir el contexto de permisos | Anadir el modulo a MODULOS y a las tablas de roles en accounts/context_processors.py                                                      |
| 9  | Definir rutas                   | urls.py con app_index (@role_required) y el DefaultRouter; registrar en ROOT_URLCONF                                                      |
| 10 | Definir formularios             | forms.py con ModelForms para crear y editar                                                                                               |
| 11 | Crear plantillas                | index.html (listado) y agregar.html / nuevo.html (formulario), Extendiendo base.html                                                      |
| 12 | Crear estilos                   | <app>/static/css/<app>.css importando variables.css; reutiliza las clases globales                                                        |
| 13 | Crear scripts                   | <app>/static/js/<app>.js; sin script inline, sin estilos inline                                                                           |
| 14 | Registrar en el admin           | admin.py con los ModelAdmin correspondientes                                                                                              |
| 15 | Probar                          | python manage.py check, luego recorrer el flujo con cada rol                                                                              |

Tabla 12.1 - Checklist para agregar un modulo.

### 12.2 Como agregar un campo a un modelo

```
# 1. Modificar el modelo
class Incident(models.Model):
    ...
    categoria_seguridad = models.CharField(max_length=50, blank=True)

# 2. Crear y aplicar la migracion
python manage.py makemigrations incidents
python manage.py migrate

# 3. Actualizar el serializer si el campo debe exponerse en la API
class IncidentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Incident
        fields = '__all__' # o la lista explicita

# 4. Si el campo debe filtrarse en la API, declararlo
filterset_fields = [..., 'categoria_seguridad']
```

**ATENCION****Cuidado con los on\_delete al agregar relaciones**

Al agregar una ForeignKey, especifique siempre on\_delete de forma explicita. CASCADE borra en cascada, PROTECT impide el borrado y SET\_NULL deja el campo vacio. Elegir por defecto puede tener consecuencias irreversibles sobre los datos.

## 12.3 Buenas practicas al escribir codigo en este proyecto

- **Idioma:** todo el codigo, comentarios, nombres de plantilla y mensajes van en espanol, salvo los nombres de clase y metodo de Django/DRF.
- **Modelos:** UUIDField como PK, related\_name explicito, on\_delete explicito, indices en los campos de filtro, unique\_together en lo que pertenece a un padre.
- **Permisos:** usar las constantes de cuentas (ROLES\_\*) y los permisos de accounts.permissions.py; nunca escribir la tupla de roles a mano.
- **Visibilidad:** todo listado debe pasar por un helper \_<entidad>visibles(user) y terminar en .distinct() cuando hay joins.
- **Serializers:** un serializer de listado ligero y uno de detalle cuando el detalle enricha; seleccionarlos con get\_serializer\_class segun la accion.
- **Frontend:** sin script inline, sin estilos inline, CSS en espanol, tokens de variables.css, rem excepto bordes y sombras.
- **JS y plantillas:** pasar datos por data-\*, nunca por etiquetas de plantilla dentro del JS.
- **Paginacion:** 15 elementos por pagina en la vista HTML; 20 en la API (el default de DRF).

## 12.4 Como diagnosticar problemas

| Sintoma                          | Causa probable                  | Que revisar                                         |
|----------------------------------|---------------------------------|-----------------------------------------------------|
| No arranca: ImproperlyConfigured | Falta DJANGO_SECRET_KEY en .env | El archivo .env y que python-dotenv lo este leyendo |

| Sintoma                             | Causa probable                                                          | Que revisar                                                   |
|-------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------|
| No arranca: No module named X       | Dependencia no instalada o app no registrada                            | pip install -r requirements.txt; INSTALLED_APPS               |
| Login correcto pero redirige mal    | status distinto de 'active'                                             | El campo status del usuario                                   |
| La API devuelve HTML en vez de JSON | El listado HTML sombrea el listado JSON                                 | El orden de las rutas en urls.py (ver 11.2)                   |
| Los estilos no cargan en produccion | collectstatic no ejecutado                                              | STATIC_ROOT y WhiteNoise                                      |
| Un filtro de la API no hace nada    | El campo no esta en filterset_fields, o se uso filter_fields (obsoleto) | Declarar el campo en filterset_fields                         |
| El JS no encuentra un valor         | El HTML no pasa el data-* esperado                                      | Revisar que el JS lea dataset y que el HTML tenga el atributo |
| Un color no coincide con el disenio | Se uso un literal en vez de una variable                                | Revisar el uso de variables.css y las reglas de px/rem        |
| Los websockets no conectan          | asgi.py no enruta websockets                                            | Cablear el ProtocolTypeRouter (capitulo 9)                    |
| check reporta 0 pero algo falla     | El bloque de configuracion que manda no es el que se lee                | Revisar duplicados en settings.py                             |

Tabla 12.2 - Diagnostico rapido por sintoma.

## 12.5 Comandos de verificacion antes de publicar

```
# 1. Verificar la configuracion (debe reportar 0 problemas)
python manage.py check

# 2. Revisar la configuracion de produccion
python manage.py check --deploy

# 3. Verificar que no hay cambios de modelo pendientes
python manage.py makemigrations --check --dry-run

# 4. Ver el estado de las migraciones
python manage.py showmigrations

# 5. Ejecutar la suite de pruebas
python manage.py test
```

## 12.6 Resumen de la arquitectura en una linea

CONDYSOS es un monolito modular de Django: un unico proyecto y una unica base de datos, dieciocho aplicaciones de dominio que se comunican exclusivamente a traves del modelo de usuario compartido y de las relaciones entre modelos, una capa de presentacion renderizada en el servidor con un sistema de disenio basado en variables, y una API REST en paralelo que expone cada modulo con los mismos datos y los mismos permisos por rol. Toda la logica de negocio vive en el lado del servidor; el JavaScript solo aporta interactividad. El sistema esta preparado para escalar hacia tiempo real y base de datos de produccion, pero ambas rutas requieren trabajo adicional documentado en los capitulos 9 y 10.